Quantum Neural Networks

Theory, Architectures, and Applications

Author: Yixiang Wang

Advisor: Ozan Tonguz, Elias Towe

Abstract

Quantum neural networks (QNNs), also known as variational or parameterized quantum circuits, are the most widely studied model in quantum machine learning. This white paper provides a self-contained, pedagogical introduction to QNNs for readers with a background in either deep learning or quantum computing. We construct a QNN from first principles and compute each of its layers explicitly on a three-qubit example: angle encoding, trainable rotations, entangling CNOT gates, and measurement. This calculation shows, in particular, how CNOT gates propagate information between qubits. We then present a step-by-step derivation of the result that a QNN computes a finite Fourier series, whose frequencies are fixed by the eigenvalues of the data-encoding Hamiltonians and whose coefficients are set by the trainable parameters. In this picture, expressivity is governed by the encoding strategy rather than by the depth of the trainable layers, and we analyze how parallel encoding, data re-uploading, and input scaling shape the accessible frequency spectrum and at what resource cost. The same Fourier structure yields the parameter-shift rule for training, which we discuss together with the barren plateau problem. Finally, we use the Fourier picture to derive criteria for task suitability and assess representative application domains against them. We conclude that QNNs are best suited to low-dimensional problems with structured spectral content, such as implicit neural representations, periodic signals, and differential equations, and, most promisingly, to quantum-native data.

1 Introduction

1.1 Deep Learning and the Cost of Scale

Over the past decade and a half, deep learning has evolved from a specialized research topic into a general-purpose technology [1]. Convolutional and residual networks transformed image recognition [2, 3], Transformers [4] underpin large language models that translate, summarize, and write code [5], deep networks predict protein structures with near-experimental accuracy [6], and graph neural networks [7] learn from relational data such as molecules and financial transaction graphs. These architectures differ in their inductive biases, yet they share a common foundation: each is a parameterized function approximator whose parameters are learned from data by gradient-based optimization [8, 9].

Much of this progress has been achieved through scale. Empirical scaling laws show that model performance improves predictably, as a power law, with model size, dataset size, and training compute [10], and the compute used to train notable models has doubled roughly every six months since the beginning of the deep learning era [11]. This growth has tangible physical costs [12]: according to the International Energy Agency, global data-center electricity consumption grew by 17% in 2025 and is projected to roughly double by 2030, while the capital expenditure of five large technology companies exceeded $400 billion in 2025, driven largely by data-center investment [13]. These trends raise a natural question: beyond building ever-larger machines that follow classical rules, are there fundamentally different models of computation that could complement them?

1.2 A Different Kind of Computer

In 1982, Feynman observed that simulating quantum systems on classical computers appears to require resources that grow exponentially with system size, and he proposed instead building computers whose components are themselves quantum mechanical [14]. Shor's algorithm factors integers in polynomial time, exponentially faster than the best known classical algorithms [15], and Grover's algorithm offers a quadratic speedup for unstructured search [16]. These algorithms draw on resources that have no classical counterpart: superposition, entanglement, and interference [17]. A register of n qubits is described by 2^n complex amplitudes, so the state space of even a modest quantum computer is far larger than the memory of any classical machine.

This exponential state space, however, does not translate directly into exponential computational power: a measurement of n qubits returns only n classical bits, and the amplitudes can be estimated only statistically through repeated measurements. This tension between a vast internal state and a narrow measurement channel recurs throughout this paper.

On the hardware side, present-day devices belong to what Preskill termed the *noisy intermediate-scale quantum* (NISQ) era: processors with tens to a few hundred qubits that operate without full error correction [18]. In 2019, a 53-qubit superconducting processor performed a random-circuit sampling task that was argued to be intractable for the classical supercomputers of the time [19]. More recently, surface-code experiments demonstrated logical error rates that *decrease* as the code distance is increased from three to five to seven, an essential prerequisite for scalable fault-tolerant quantum computation [20]. Until large-scale fault-tolerant machines arrive, *variational quantum algorithms* offer a way to use noisy devices of moderate size [21]: a parameterized quantum circuit is executed on quantum hardware, and its parameters are optimized by a classical computer in a hybrid feedback loop.

1.3 Quantum Neural Networks

Quantum machine learning lies at the intersection of these two developments [22, 23], and its most widely studied model is the *quantum neural network* (QNN), also known as a variational or parameterized quantum circuit [24, 25, 26]. The structure of a QNN deliberately mirrors that of a classical neural network. Classical input data $\mathbf{x}$ are encoded into a quantum state, typically as rotation angles; layers of trainable rotations and entangling gates transform this state; and the expectation value of an observable O is measured to produce a classical output:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \langle 0|^{\otimes n}\, U^{\dagger}(\mathbf{x},\boldsymbol{\theta})\, O\, U(\mathbf{x},\boldsymbol{\theta})\, |0\rangle^{\otimes n}\,. \tag{1.1}$$

The model is trained in the same way as its classical counterpart: a loss function is defined, its gradients with respect to the circuit parameters $\boldsymbol{\theta}$ are estimated on the quantum device [25] (Section 4.4), and the parameters are updated by a classical optimizer.

QNNs can use the exponentially large Hilbert space as a feature space [27, 28]; a single qubit that repeatedly re-encodes its input can act as a universal classifier [29]; and certain QNNs have been shown to possess a larger effective dimension than comparable classical networks [30]. At the same time, the field faces serious open questions. Randomly initialized circuits can exhibit *barren plateaus*, in which gradients vanish exponentially with the number of qubits and training stalls [31]. Classical learners that have access to data can often match

quantum models [32], many QNNs can be closely approximated by classical random-Fourier-feature models [33], and the very structure that allows a circuit to avoid barren plateaus may also allow it to be simulated classically [34]. Whether QNNs offer a practical advantage on classical data therefore remains open, and a more productive starting point is a more fundamental question: *what kind of functions does a QNN compute, and how is this function class shaped by the design of the circuit?*

1.4 A Fourier Perspective

This question is the central theme of this paper, and Fourier analysis provides a remarkably clear answer. Schuld, Sweke, and Meyer [35] showed that, when a scalar input x enters the circuit through gates generated by a Hamiltonian, the output of a QNN is a finite Fourier series:

$$f_{\boldsymbol{\theta}}(x) = \sum_{\omega \in \Omega} c_\omega(\boldsymbol{\theta})\, e^{i\omega x}. \tag{1.2}$$

This representation reveals a clean division of labor. The set of frequencies Ω is determined entirely by the eigenvalues of the encoding Hamiltonians and by how many times the input is encoded; the trainable parameters $\boldsymbol{\theta}$ only shape the coefficients c_ω. The consequences are striking. A single qubit that encodes x once through the rotation $R_x(x)$ can represent only a single sinusoid, $c_0 + 2|c_1|\cos(x + \varphi)$, no matter how many trainable layers surround the encoding gate. The expressivity of a QNN is therefore governed less by the depth of its trainable layers than by *how the data are encoded*, which makes the encoding strategy (the number of encoding qubits, the number of re-uploading layers, and the scaling of the input) the principal design lever.

This perspective also invites a direct comparison with classical networks. Multilayer perceptrons exhibit a *spectral bias*: they learn low-frequency components first and struggle to fit high-frequency detail [36]. Classical remedies, such as Fourier feature mappings [37] and periodic activation functions as in SIREN [38], effectively allow the network to learn or select the frequencies it uses. A QNN inverts this arrangement: its frequencies are fixed in advance by the architecture, chosen by the designer rather than discovered during training, and learning adjusts only their amplitudes and phases. Is such a predetermined spectrum a fundamental limitation, since the model can never represent a frequency outside Ω, or is it an advantage, since it provides an explicit, interpretable, and controllable spectral inductive bias? This question runs throughout the paper.

1.5 Scope and Organization

This white paper is intended as a self-contained, pedagogical bridge between classical deep learning and quantum computing for readers familiar with one field and new to the other. Rather than surveying the literature, we construct QNNs from first principles and work through small examples explicitly, so that every claim can be verified step by step. In particular, this paper:

- frames QNNs and multilayer perceptrons as two families of parameterized function approximators, using the universal approximation theorem as a common reference point;

- dissects a standard QNN layer by layer (encoding, trainable rotations, entanglement, and measurement), showing explicitly, for example, how CNOT gates propagate information between qubits;
- presents a complete, step-by-step derivation of the finite Fourier series representation of QNNs;
- analyzes how parallel encoding, data re-uploading, and input scaling shape the accessible frequency spectrum, together with the resource trade-offs among them;
- derives the parameter-shift rule for training directly from the Fourier representation and discusses the barren plateau problem; and
- derives criteria for task suitability from the Fourier picture and assesses representative application domains against them.

The remainder of the paper is organized as follows. Section 2 reviews multilayer perceptrons and the universal approximation theorem. Section 3 introduces the essentials of quantum computing and dissects the architecture of a QNN. Section 4 develops the Fourier analysis of QNNs, examines how the encoding strategy determines their expressivity, and briefly discusses training. Section 5 discusses applications and limitations, and Section 6 concludes.

2 Review of Classical Neural Networks

Before exploring Quantum Neural Networks (QNNs), it is essential to understand their classical counterparts. At their core, neural networks are parameterized mathematical models designed to approximate complex functions from data. Among the various architectures, the Multilayer Perceptron (MLP) is one of the most fundamental and widely studied. Despite its relatively simple structure, the MLP provides the theoretical foundation for understanding the expressive power of neural networks.

2.1 Architecture of Multilayer Perceptrons

An MLP is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer, as illustrated in Figure 1 [9, Chap. 6]. Each hidden layer consists of multiple neurons, which perform affine transformations followed by nonlinear activation functions; the output layer is typically affine.

Mathematically, the output of the l-th hidden layer can be expressed as

$$\mathbf{h}^{(l)} = \sigma\left(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \tag{2.1}$$

where $\mathbf{h}^{(0)} = \mathbf{x}$ denotes the input vector, $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the trainable weight matrix and bias vector, respectively, and σ is a nonlinear activation function. Common choices include the sigmoid, the hyperbolic tangent (tanh), and the rectified linear unit (ReLU).

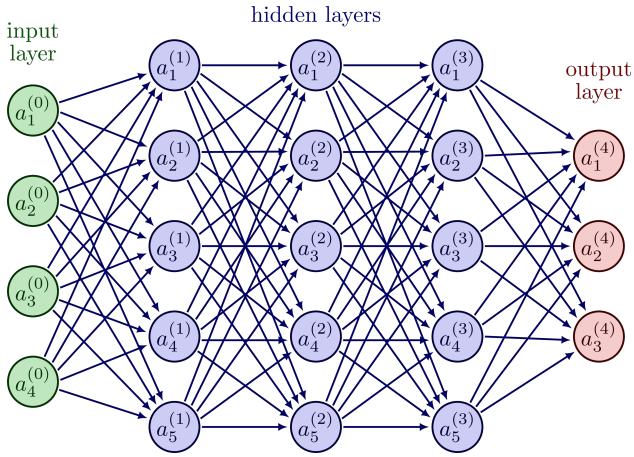


Figure 1: Architecture of a Multilayer Perceptron (MLP), consisting of an input layer, hidden layers, and an output layer.

By composing multiple layers, an MLP constructs a parameterized mapping from the input space to the output space. Let $A^{(l)}(\mathbf{h}) = W^{(l)}\mathbf{h} + \mathbf{b}^{(l)}$ denote the affine transformation at layer l. An MLP with L affine layers and a linear output layer can then be represented as

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = A^{(L)} \circ \sigma \circ A^{(L-1)} \circ \cdots \circ \sigma \circ A^{(1)}(\mathbf{x}), \tag{2.2}$$

where $\boldsymbol{\theta}$ represents all trainable parameters and $\circ$ denotes function composition.

An important observation is that the nonlinear activation function is essential. Without nonlinearities, any composition of affine transformations remains an affine transformation, regardless of the number of layers. Therefore, nonlinear activations are what enable MLPs to represent functions beyond simple linear mappings.

2.2 MLPs as Function Approximators

The fundamental objective of an MLP is to approximate an unknown target function $f^*(\mathbf{x})$ using a parameterized function $f_{\boldsymbol{\theta}}(\mathbf{x})$.

To understand how this works, consider a single-hidden-layer MLP with a scalar output:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=1}^{m} a_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j) + c, \tag{2.3}$$

where m is the number of hidden neurons, $\mathbf{w}_j$ and b_j determine the transformation applied by the j-th neuron, and a_j controls its contribution to the final output.

This expression reveals a fundamental principle: an MLP approximates a complex function by combining multiple simpler nonlinear functions. Each hidden neuron generates a parameterized basis-like function, and the output layer constructs a weighted combination of these functions.

For example, consider the ReLU activation:

$$\sigma(x) = \max(0, x). \tag{2.4}$$

Each ReLU neuron introduces a piecewise-linear component. By combining sufficiently many such components, an MLP can construct increasingly complex piecewise-linear functions that approximate nonlinear target functions, as illustrated in Figure 2.

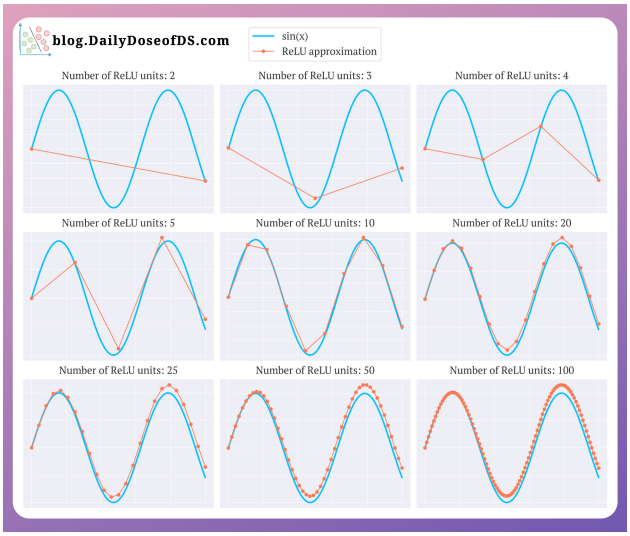


Figure 2: Illustration of function approximation using ReLU-based MLPs. Increasing the number of hidden neurons provides greater flexibility to approximate a nonlinear target function.

Given a training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the network learns its parameters by minimizing a loss function. For example, in regression tasks, the parameters are obtained by minimizing the mean squared error (MSE):

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \left(f_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i \right)^2 . \tag{2.5}$$

In practice, the parameters are typically optimized through gradient-based methods using backpropagation [8].

2.3 Universal Approximation Theorem

A natural question arises: how complex a function can an MLP approximate?

One of the most important theoretical results in neural network research is the Universal Approximation Theorem (UAT), established through a series of foundational works [39, 40].

In particular, consider a continuous target function $f^* : K \to \mathbb{R}$, where $K \subset \mathbb{R}^d$ is compact. For any approximation tolerance $\epsilon > 0$, there exists a single-hidden-layer ReLU network with sufficiently many hidden neurons such that

$$\boxed{\sup_{\mathbf{x} \in K} |f^*(\mathbf{x}) - f_{\boldsymbol{\theta}}(\mathbf{x})| < \epsilon}. \tag{2.6}$$

In other words, even an MLP with only one hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy, provided that sufficiently many hidden neurons are available.

This theorem establishes the remarkable expressive potential of neural networks. However, several important distinctions must be made.

First, the theorem guarantees the *existence* of suitable parameters but does not guarantee that a training algorithm can efficiently discover them. Second, it does not specify how many neurons or parameters are required to achieve a desired approximation accuracy. Finally, approximation capability does not necessarily imply good generalization to unseen data.

Although a single hidden layer is theoretically sufficient for universal approximation, deeper architectures may represent certain functions more efficiently by exploiting their compositional structure [9, 41].

2.4 Beyond MLPs: Toward Quantum Neural Networks

Modern deep learning has extended the basic principles of MLPs into numerous specialized architectures. Convolutional Neural Networks (CNNs) exploit spatial structure; recurrent architectures such as RNNs, LSTMs, and GRUs model sequential dependencies; Transformers leverage attention mechanisms; and Graph Neural Networks (GNNs) incorporate graph structure [9, 4, 7]. Despite their architectural differences, these models share the fundamental objective of learning parameterized functions from data.

This naturally leads to an intriguing question: can quantum computers also serve as function approximators?

While classical MLPs construct functions through compositions of affine transformations and nonlinear activations, variational QNNs construct functions through quantum data encoding, parameterized unitary transformations, and measurements. As previewed in Eq. (1.1), a QNN produces its output as the expectation value of a measurement observable O in the state prepared by a parameterized quantum circuit $U(\mathbf{x}, \boldsymbol{\theta})$ [25, 26].

Thus, both MLPs and QNNs can be viewed as parameterized function approximators, yet they rely on fundamentally different computational mechanisms. This raises several important questions: How do QNNs represent functions? What determines their expressive power? And how does their function approximation capability compare with that of classical MLPs?

These questions motivate the following investigation into the mathematical representation and expressive properties of Quantum Neural Networks.

3 Quantum Neural Networks

3.1 Basics of Quantum Computing

Quantum computing is built upon the principles of quantum mechanics, which provide a fundamentally different computational framework from classical computing. Unlike classical computers, which process information using binary bits, quantum computers manipulate quantum bits, or qubits, through unitary transformations and quantum measurements. This section introduces the essential concepts of quantum computing, including quantum states, quantum gates, tensor products, entanglement, and measurement. These concepts form the

mathematical foundation of Quantum Neural Networks (QNNs). The following discussion is based on the standard quantum computing formalism presented in [17, Chaps. 2 and 4].

3.1.1 Qubits and Quantum States

In classical computing, a bit can take one of two possible values, either 0 or 1. In contrast, a qubit can exist in a superposition of two computational basis states, denoted by $|0\rangle$ and $|1\rangle$. The general state of a single qubit is expressed as

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |\alpha|^2 + |\beta|^2 = 1, \tag{3.1}$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes. In vector notation,

$$|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}, \quad |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}, \quad |\psi\rangle = \begin{bmatrix}\alpha\\\beta\end{bmatrix}. \tag{3.2}$$

According to the Born rule, the probabilities of measuring $|0\rangle$ and $|1\rangle$ are $|\alpha|^2$ and $|\beta|^2$, respectively. Importantly, the probability amplitudes also contain phase information, which plays a crucial role in quantum interference.

Ignoring an unobservable global phase, any pure single-qubit state can be represented geometrically on the Bloch sphere, as illustrated in Figure 3:

$$|\psi\rangle = \cos\left(\frac{\vartheta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\vartheta}{2}\right)|1\rangle\,, \tag{3.3}$$

where ϑ and φ represent the polar and azimuthal angles, respectively.

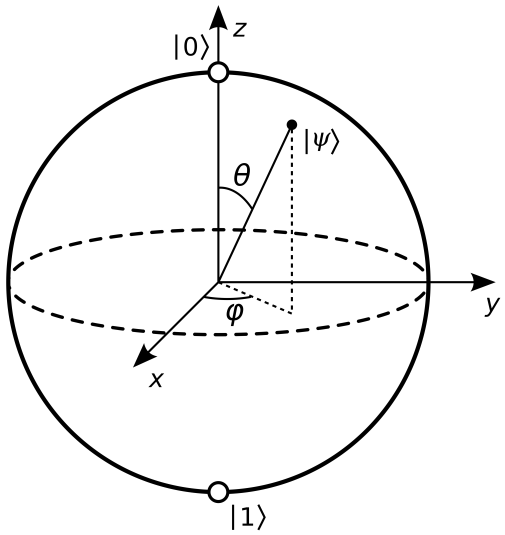


Figure 3: Bloch sphere representation of a single-qubit pure state. The angles ϑ and φ characterize its position on the sphere, up to a global phase.

For a system containing n qubits, the quantum state belongs to a 2^n-dimensional Hilbert space and can be expressed as

$$|\psi\rangle = \sum_{j=0}^{2^n-1} a_j\,|j\rangle\,, \qquad \sum_{j=0}^{2^n-1} |a_j|^2 = 1, \tag{3.4}$$

where $|j\rangle$ denotes the computational basis state corresponding to the binary representation of j.

Throughout this paper, we adopt the *little-endian convention*, in which the computational basis is ordered as $|q_{n-1}\cdots q_1 q_0\rangle$, with q_0 being the least significant bit.

Although the dimension of the quantum state space grows exponentially with the number of qubits, this does not automatically imply an exponential computational advantage, since extracting information from a quantum state is constrained by quantum measurement.

3.1.2 Quantum Gates and Unitary Operations

Quantum states are manipulated through quantum gates, which are mathematically represented by unitary matrices. A quantum gate U transforms an initial state $|\psi\rangle$ into

$$|\psi'\rangle = U|\psi\rangle, \qquad U^\dagger U = I, \tag{3.5}$$

where $U^\dagger$ is the conjugate transpose of U. The unitary condition ensures that the normalization of the quantum state is preserved throughout the evolution of an ideal quantum circuit.

Several fundamental single-qubit gates are widely used in quantum computing. For example, the Pauli-X gate acts similarly to a classical NOT gate, while the Hadamard gate transforms computational basis states into equal superpositions:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}. \tag{3.6}$$

In particular,

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle). \tag{3.7}$$

Another important class of gates consists of the rotation gates, which rotate quantum states around different axes of the Bloch sphere. They can be expressed as

$$R_\nu(\theta) = e^{-i\theta\sigma_\nu/2}, \qquad \nu \in \{x, y, z\}, \tag{3.8}$$

where σ_ν denotes the corresponding Pauli operator; their explicit matrices are given in Section 3.2. Rotation gates are particularly important in QNNs because their rotation angles can encode classical input features or represent trainable model parameters.

When multiple qubits are involved, the combined state space is constructed using the tensor product. If independent single-qubit gates $U_0, U_1, \ldots, U_{n-1}$ act on their respective qubits, the overall operation is represented, under our little-endian convention, as

$$U = U_{n-1} \otimes \cdots \otimes U_1 \otimes U_0. \tag{3.9}$$

The tensor product provides a systematic way to construct multi-qubit operations from individual gates. However, not all multi-qubit operations can be decomposed into tensor products of single-qubit gates. Such operations are essential for generating quantum entanglement.

3.1.3 Multi-Qubit Gates and Entanglement

One of the most fundamental multi-qubit gates is the Controlled-NOT (CNOT) gate, which operates on two qubits: a control qubit and a target qubit. The target is flipped if and only if the control is in state $|1\rangle$. Its action on computational basis states is defined as

$$|c, t\rangle \xrightarrow{\text{CNOT}} |c, t \oplus c\rangle\,, \tag{3.10}$$

where $\oplus$ denotes modulo-2 addition.

For a two-qubit system following the little-endian convention $|q_1 q_0\rangle$, if q_1 is the control and q_0 is the target, the corresponding matrix is

$$\text{CNOT}_{q_1 \to q_0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}. \tag{3.11}$$

For example, $\text{CNOT}_{q_1 \to q_0} |10\rangle = |11\rangle$.

Conversely, if q_0 is the control and q_1 is the target, the matrix becomes

$$\text{CNOT}_{q_0 \to q_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \tag{3.12}$$

Notice that reversing the control and target qubits changes the matrix representation. For instance,

$$\text{CNOT}_{q_0 \to q_1} |01\rangle = |11\rangle\,. \tag{3.13}$$

Beyond conditional bit flipping, the CNOT gate plays a crucial role in creating quantum entanglement. Consider two qubits initially prepared in $|00\rangle$. As illustrated in Figure 4, we first apply a Hadamard gate to q_0:

$$\begin{aligned} |\psi\rangle &= (I \otimes H)\,|00\rangle \\ &= \frac{1}{\sqrt{2}}\left(|00\rangle + |01\rangle\right). \end{aligned} \tag{3.14}$$

Next, applying a CNOT gate with q_0 as the control and q_1 as the target produces

$$\begin{aligned} |\psi_{\text{Bell}}\rangle &= \text{CNOT}_{q_0 \to q_1}\,|\psi\rangle \\ &= \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right). \end{aligned} \tag{3.15}$$

The resulting Bell state is entangled, meaning that it cannot be expressed as the tensor product of two individual single-qubit states. Measuring both qubits in the computational basis produces either $|00\rangle$ or $|11\rangle$, each with probability $1/2$.

More generally, for an arbitrary two-qubit pure state

$$|\psi\rangle = a\,|00\rangle + b\,|01\rangle + c\,|10\rangle + d\,|11\rangle\,, \tag{3.16}$$

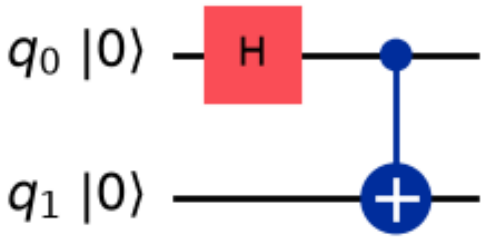


Figure 4: Generation of a Bell state using a Hadamard gate followed by a CNOT gate. The resulting two-qubit state is maximally entangled.

entanglement can be determined by examining the coefficient matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}. \tag{3.17}$$

The state is separable if and only if A has rank one or, equivalently, if its determinant vanishes:

$$\boxed{\begin{cases} ad - bc = 0, & \text{Separable} \\ ad - bc \neq 0, & \text{Entangled} \end{cases}}. \tag{3.18}$$

This criterion applies specifically to two-qubit pure states. For the Bell state in Eq. (3.15), $ad - bc = 1/2 \neq 0$, confirming that the state is entangled.

Entangling gates such as CNOT are important building blocks of QNNs, as they enable quantum circuits to establish interactions among qubits that cannot be generated by independent single-qubit operations alone.

3.1.4 Quantum Measurement

Unlike in classical computation, where the values of computational variables can generally be accessed directly, extracting information from a quantum system requires measurement. A computational-basis measurement of the qubit in Eq. (3.1) yields 0 or 1 with the Born-rule probabilities $|\alpha|^2$ and $|\beta|^2$, and a single measurement does not reveal the underlying complex amplitudes. Instead, repeated preparation and measurement of the same quantum state allow us to estimate measurement probabilities and expectation values statistically. In particular, the expectation value of a Hermitian observable O for a quantum state $|\psi\rangle$ is

$$\langle O \rangle = \langle \psi | \, O \, | \psi \rangle \, . \tag{3.19}$$

This provides a natural mechanism for converting quantum information into classical numerical outputs, and it is how a QNN produces its output in Eq. (1.1). Section 3.7 treats measurement in detail.

Thus, quantum states, unitary operations, entanglement, and measurement collectively provide the building blocks of QNNs: classical input data are encoded into quantum states, trainable gates transform these states, entangling operations introduce interactions among qubits, and measurements produce classical outputs. The following subsections examine how these building blocks are combined into a QNN.

3.2 Dissection of a Quantum Neural Network

Like their classical counterparts, QNNs serve as parameterized function approximators, and with suitable data encodings and circuit families they can approximate broad classes of functions [25, 29, 35]. Before analyzing what such a model computes, we examine how it is built. Figure 5 shows a representative three-qubit QNN. The circuit consists of an encoding layer that loads the classical input into the qubits, followed by layers of parameterized single-qubit rotation gates (R_x, R_y, R_z) and entangling CNOT gates. Each qubit is finally measured in the computational (Z) basis. Remarkably, these two kinds of gates, single-qubit rotations and CNOT, suffice to build every QNN considered in this paper. We therefore first review them in the form needed for the subsequent analysis and then outline how they are arranged into layers.

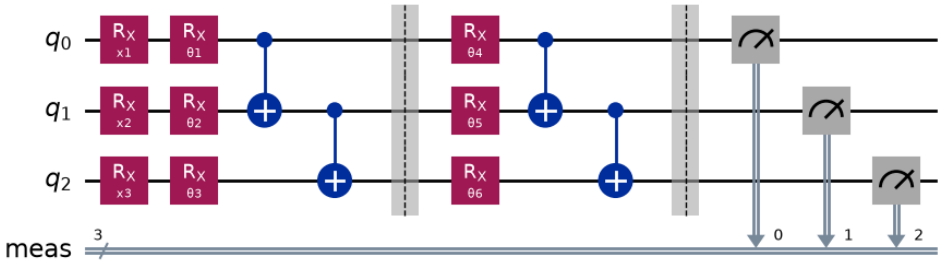


Figure 5: A representative three-qubit QNN: an encoding layer, layers of parameterized rotation gates and CNOT gates, and a measurement of each qubit in the computational basis.

Rotation gates. Every single-qubit gate is a 2×2 unitary matrix. Writing out Eq. (3.8) for each axis, the rotation gates R_x, R_y, and R_z are generated by the Pauli matrices X, Y, and Z, respectively:

$$R_x(\theta) \equiv e^{-i\theta X/2} = \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) X = \begin{bmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix}, \tag{3.20}$$

$$R_y(\theta) \equiv e^{-i\theta Y/2} = \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) Y = \begin{bmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix}, \tag{3.21}$$

$$R_z(\theta) \equiv e^{-i\theta Z/2} = \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) Z = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}. \tag{3.22}$$

The second equality in each line follows from the Taylor series of the exponential, because every Pauli matrix squares to the identity: even powers of $P \in \{X, Y, Z\}$ reduce to I and contribute the cosine, while odd powers reduce to P and contribute the sine. Geometrically, $R_x(\theta)$, $R_y(\theta)$, and $R_z(\theta)$ rotate the Bloch vector of a qubit by an angle θ (in radians) about the x-, y-, and z-axes of the Bloch sphere, respectively. The factor $\frac{1}{2}$ in the exponent reflects the half-angle parameterization of the Bloch sphere in Eq. (3.3): a change of θ in the gate angle corresponds to a change of $\theta/2$ in the amplitudes and hence to a rotation by θ on the sphere. Rotation gates play two distinct roles in a QNN. When the angle is set by an input feature, $\theta = x$, the gate encodes data; when the angle is a trainable parameter, the gate plays the role of a weight in a classical neural network.

The CNOT gate. As introduced in Section 3.1.3, the CNOT gate flips its target qubit if and only if its control qubit is in state $|1\rangle$ (Eq. (3.10)). Its matrix representation depends on the ordering of the basis. Under the little-endian convention, the two-qubit basis state $|q_1 q_0\rangle$ corresponds to the basis vector with index $2q_1 + q_0$, so that with q_0 as the control and q_1 as the target the CNOT matrix is that of Eq. (3.12). It leaves $|00\rangle$ and $|10\rangle$, the states with $q_0 = 0$, unchanged and exchanges $|01\rangle$ and $|11\rangle$, the states with $q_0 = 1$; for example, $\mathrm{CNOT}_{q_0 \to q_1} |11\rangle = |01\rangle$. As shown in Section 3.1.3, it is precisely this conditional action that allows CNOT to create entanglement, which single-qubit gates alone cannot do.

Universality. Rotation and CNOT gates together form a universal gate set, playing a role analogous to that of the NAND gate in classical logic. Up to a global phase, any single-qubit unitary can be written as a product of rotation gates, for instance as $R_z(\alpha)R_y(\beta)R_z(\gamma)$ [17, Sec. 4.2], and any multi-qubit unitary can be decomposed exactly into single-qubit gates and CNOT gates [42]. Such a decomposition may, however, require a number of gates that grows exponentially with the number of qubits for a general unitary. Universality therefore guarantees that the building blocks of a QNN are sufficient in principle, but not that every unitary can be realized by a shallow circuit.

From gates to layers. The structure of a QNN mirrors the input, hidden, and output layers of a classical neural network:

- the *input layer* encodes the classical data into the quantum state, typically through rotation gates whose angles are the input features (Section 3.3);
- the *hidden layers* transform the state in Hilbert space through trainable rotation gates (Section 3.4) and entangling CNOT gates (Section 3.5); and
- the *output layer* measures the qubits and converts the quantum state into classical outputs (Section 3.7).

The following subsections examine each of these layers in turn.

3.3 Input Layer: Encoding

In most machine learning tasks the data are classical, whereas a quantum circuit operates on quantum states. The first layer of a QNN must therefore *encode* each classical input into a quantum state. The choice of encoding is not a mere technicality: as shown in Section 4, it determines which functions the model can represent. Three basic encoding strategies are commonly distinguished [26, 23]:

1. **Basis encoding** maps a binary string $\mathbf{b} = (b_{n-1}, \ldots, b_0)$ to the computational basis state $|b_{n-1} \cdots b_0\rangle$, using one qubit per bit. It is natural for discrete data but cannot represent continuous values directly.

2. **Amplitude encoding** stores a normalized vector $\mathbf{x} \in \mathbb{C}^{2^n}$ in the 2^n amplitudes of an n-qubit state, $|\psi\rangle = \sum_j x_j |j\rangle$. It is exponentially compact in the number of qubits, but preparing an arbitrary such state generally requires deep circuits.

3. **Angle encoding** (also called angular encoding) uses each feature as the rotation angle of a single-qubit gate, with one qubit per feature.

More elaborate schemes, such as encodings based on products of Pauli rotations and on circuits of the instantaneous quantum polynomial (IQP) type [27], are beyond the scope of this paper. We focus on angle encoding because it is the most widely used encoding in variational QNNs and the simplest to implement: it requires only a single layer of single-qubit gates, so it adds a circuit depth of one regardless of the number of features.

Single-qubit angle encoding. Figure 6 shows a three-qubit circuit that encodes the three-dimensional input $\mathbf{x} = (x_1, x_2, x_3)$. Each qubit, represented by a horizontal line, is initialized in the state $|0\rangle$ and then passes through a rotation gate whose angle is one of the input features. Following the little-endian convention, feature x_1 is encoded on qubit q_0, x_2 on q_1, and x_3 on q_2.

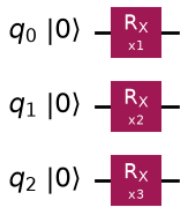


Figure 6: Input layer of a QNN using angle encoding: feature x_{k+1} sets the angle of a rotation gate acting on qubit q_k.

Consider first the qubit q_0, which starts in $|0\rangle = (1, 0)^T$. Using the gate R_x of Eq. (3.20) with angle x_1, we obtain

$$R_x(x_1)\,|0\rangle = \begin{bmatrix} \cos\frac{x_1}{2} & -i\sin\frac{x_1}{2} \\ -i\sin\frac{x_1}{2} & \cos\frac{x_1}{2} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos\frac{x_1}{2} \\ -i\sin\frac{x_1}{2} \end{bmatrix} = \cos\left(\frac{x_1}{2}\right)|0\rangle - i\sin\left(\frac{x_1}{2}\right)|1\rangle\,. \quad (3.23)$$

Applying a gate to $|0\rangle$ simply selects its first column. With R_y of Eq. (3.21) instead, the encoded state is

$$R_y(x_1)\,|0\rangle = \cos\!\left(\frac{x_1}{2}\right)|0\rangle + \sin\!\left(\frac{x_1}{2}\right)|1\rangle\,, \tag{3.24}$$

which has real amplitudes. The two encodings yield the same measurement probabilities in the computational basis, $P(0) = \cos^2(x_1/2)$ and $P(1) = \sin^2(x_1/2)$, and differ only in the relative phase between $|0\rangle$ and $|1\rangle$. The gate R_z, by contrast, is not suitable as the first encoding gate: since $|0\rangle$ is an eigenstate of Z, Eq. (3.22) gives $R_z(x_1)\,|0\rangle = e^{-ix_1/2}\,|0\rangle$, which differs from $|0\rangle$ only by a global phase and therefore carries no information about x_1. An R_z encoding becomes informative only when it is preceded by a gate that creates a superposition, such as a Hadamard gate.

In the remainder of this section we use R_x. With the shorthand

$$c_j = \cos\!\left(\frac{x_j}{2}\right), \qquad s_j = \sin\!\left(\frac{x_j}{2}\right), \qquad j = 1, 2, 3, \tag{3.25}$$

the three qubits are encoded as

$$|q_0\rangle = \begin{bmatrix} c_1 \\ -is_1 \end{bmatrix}, \qquad |q_1\rangle = \begin{bmatrix} c_2 \\ -is_2 \end{bmatrix}, \qquad |q_2\rangle = \begin{bmatrix} c_3 \\ -is_3 \end{bmatrix}. \tag{3.26}$$

The three-qubit encoded state. Because each encoding gate acts on a single qubit and no entangling gate has yet been applied, the three-qubit state is a *product state*: the tensor product of the individual qubit states. Under the little-endian convention, the state is written $|q_2 q_1 q_0\rangle = |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$, so the encoding maps the initial state $|000\rangle = (1,0,0,0,0,0,0,0)^T$ to

$$|\psi(\mathbf{x})\rangle = \begin{bmatrix} c_3 \\ -is_3 \end{bmatrix} \otimes \begin{bmatrix} c_2 \\ -is_2 \end{bmatrix} \otimes \begin{bmatrix} c_1 \\ -is_1 \end{bmatrix} = \begin{bmatrix} c_3c_2c_1 \\ -i\,c_3c_2s_1 \\ -i\,c_3s_2c_1 \\ -c_3s_2s_1 \\ -i\,s_3c_2c_1 \\ -s_3c_2s_1 \\ -s_3s_2c_1 \\ i\,s_3s_2s_1 \end{bmatrix} \begin{matrix} \leftarrow |000\rangle \\ \leftarrow |001\rangle \\ \leftarrow |010\rangle \\ \leftarrow |011\rangle \\ \leftarrow |100\rangle \\ \leftarrow |101\rangle \\ \leftarrow |110\rangle \\ \leftarrow |111\rangle \end{matrix} \tag{3.27}$$

The tensor product of three two-dimensional vectors is an eight-dimensional vector, consistent with the state space $\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^8$. Each amplitude is a product of one factor per qubit: c_j if the corresponding qubit is in $|0\rangle$, and $-is_j$ if it is in $|1\rangle$. For example, the amplitude of $|011\rangle$ is $c_3 \cdot (-is_2) \cdot (-is_1) = -c_3s_2s_1$.

The encoding unitary. The same structure holds at the level of operators. On a single qubit the encoding is a 2×2 matrix; on the three-qubit register it is the $2^3 \times 2^3 = 8 \times 8$ matrix

$$U_{\text{enc}}(\mathbf{x}) = R_x(x_3) \otimes R_x(x_2) \otimes R_x(x_1), \tag{3.28}$$

where, consistently with the little-endian convention, the factor acting on q_2 appears on the left and the factor acting on q_0 on the right. Written out explicitly,

$$U_{\text{enc}}(\mathbf{x}) = \begin{bmatrix} c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 \\ -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 \\ -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 \\ -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 \\ -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 \\ -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 \\ -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 \\ is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 \end{bmatrix}. \tag{3.29}$$

Since the initial state $|000\rangle$ is the first basis vector, $U_{\text{enc}}(\mathbf{x})\,|000\rangle$ is the first column of this matrix, which reproduces Eq. (3.27).

Although the matrix appears complicated, it follows a simple rule. Each factor $R_x(x_j)$ has diagonal entries c_j and off-diagonal entries $-is_j$, and every entry of a tensor product is the product of one entry from each factor. Writing a basis state as $|\mathbf{b}\rangle = |b_3b_2b_1\rangle$, where b_j denotes the state of the qubit that encodes x_j, the entry in row $\mathbf{b}$ and column $\mathbf{b}'$ is therefore

$$\big[U_{\text{enc}}(\mathbf{x})\big]_{\mathbf{b},\mathbf{b}'} = (-i)^{d_H(\mathbf{b},\mathbf{b}')} \prod_{j\,:\,b_j=b'_j} c_j \prod_{j\,:\,b_j\neq b'_j} s_j, \tag{3.30}$$

where $d_H(\mathbf{b},\mathbf{b}')$ is the Hamming distance between $\mathbf{b}$ and $\mathbf{b}'$, that is, the number of qubits whose state differs between the two basis states. Each qubit that is left unchanged contributes a cosine, and each qubit that is flipped contributes a sine together with a factor $-i$.

Feature scaling. Because the encoding depends on the features only through trigonometric functions, the range of the input values matters. The encoding gate is periodic up to a sign: $R_x(x+2\pi) = -R_x(x)$, so inputs that differ by 2π produce states that differ only by a global phase and are physically indistinguishable. In addition, the computational-basis measurement probabilities $P(0) = \cos^2(x/2) = \frac{1}{2}(1+\cos x)$ and $P(1) = \sin^2(x/2)$ are symmetric about $x = \pi$, so the inputs x and $2\pi - x$ yield identical statistics when the qubit is measured directly. Unscaled features with a wide range would therefore be mapped onto overlapping, and possibly indistinguishable, encodings.

A common remedy is to rescale each feature to the interval $[0,\pi]$ before encoding, for example by min–max normalization,

$$\tilde{x} = \pi\,\frac{x - x_{\min}}{x_{\max} - x_{\min}}, \tag{3.31}$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum of the feature over the training data; values outside this range at test time are typically clipped. On $[0,\pi]$, the expectation value $\langle Z\rangle = \cos\tilde{x}$ decreases monotonically from $+1$ to -1, so distinct inputs produce distinct measurement statistics, and the endpoints are mapped to the orthogonal states $|0\rangle$ and $|1\rangle$ (up to a phase), which are maximally distinguishable. This interval is a convention rather than a strict requirement. As shown in Section 4.2, the model output is periodic in each input, and restricting the inputs to a sub-interval of one period also prevents the model from being forced to take equal values at the two ends of the data range.

Beyond basic angle encoding. Many variants of angle encoding exist. Data re-uploading encodes the same features repeatedly between trainable layers [29]; scaled encodings multiply each feature by a constant before encoding (Section 4.3.3); and trainable encodings learn these coefficients during training [43]. As Section 4.3 shows, such choices can substantially change the expressive power of the model [35].

3.4 Hidden Layer: Trainable Rotations

After the input layer, the circuit applies its first *hidden layer*: a trainable rotation gate on each qubit, as shown in Figure 7. The angles $\theta_1, \theta_2, \theta_3$ of these gates are the trainable parameters of the layer and are adjusted during training, much like the parameters of a classical neural network. In this subsection we compute the state produced by this layer and examine what kind of parameter it introduces.

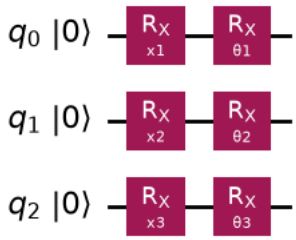


Figure 7: First hidden layer: a trainable rotation gate $R_x(\theta_{k+1})$ is applied to each qubit q_k after the encoding layer.

A single qubit. By Eq. (3.23), the encoding layer leaves qubit q_0 in the state

$$|q_0\rangle = R_x(x_1)\,|0\rangle = \cos\!\left(\frac{x_1}{2}\right)|0\rangle - i\sin\!\left(\frac{x_1}{2}\right)|1\rangle = \begin{bmatrix} \cos\frac{x_1}{2} \\ -i\sin\frac{x_1}{2} \end{bmatrix}. \tag{3.32}$$

Suppose the trainable gate is also an R_x rotation, with angle θ_1. Multiplying the matrix of Eq. (3.20) by this vector gives

$$R_x(\theta_1)\,|q_0\rangle = \begin{bmatrix} \cos\frac{\theta_1}{2} & -i\sin\frac{\theta_1}{2} \\ -i\sin\frac{\theta_1}{2} & \cos\frac{\theta_1}{2} \end{bmatrix} \begin{bmatrix} \cos\frac{x_1}{2} \\ -i\sin\frac{x_1}{2} \end{bmatrix} = \begin{bmatrix} \cos\frac{\theta_1}{2}\cos\frac{x_1}{2} - \sin\frac{\theta_1}{2}\sin\frac{x_1}{2} \\ -i\left(\sin\frac{\theta_1}{2}\cos\frac{x_1}{2} + \cos\frac{\theta_1}{2}\sin\frac{x_1}{2}\right) \end{bmatrix}, \tag{3.33}$$

where we used $(-i)(-i) = -1$ in the first entry. By the angle-addition identities

$$\cos(A+B) = \cos A\cos B - \sin A\sin B, \qquad \sin(A+B) = \sin A\cos B + \cos A\sin B, \tag{3.34}$$

applied with $A = \theta_1/2$ and $B = x_1/2$, the result simplifies to

$$R_x(\theta_1)\,R_x(x_1)\,|0\rangle = \begin{bmatrix} \cos\frac{\theta_1+x_1}{2} \\ -i\sin\frac{\theta_1+x_1}{2} \end{bmatrix} = R_x(\theta_1+x_1)\,|0\rangle\,. \tag{3.35}$$

This result has a simple explanation. Two rotations about the same axis combine into a single rotation by the sum of their angles: since X commutes with itself,

$$R_x(\theta)\,R_x(x) = e^{-i\theta X/2}\,e^{-ixX/2} = e^{-i(\theta+x)X/2} = R_x(\theta+x). \tag{3.36}$$

On the Bloch sphere, the encoding gate rotates the state by x_1 about the x-axis, and the trainable gate continues the rotation about the same axis by a further θ_1.

Interpretation: a bias rather than a weight. Equation (3.35) shows that the trainable angle enters the state only through the sum $x_1 + \theta_1$. In the language of classical neural networks, where a neuron computes $\sigma(wx + b)$, the parameter θ_1 therefore acts as a *bias*: it shifts the input, but it does not rescale it as a weight would. This observation anticipates a central result of Section 4.2. Trainable gates can shift and reweight the oscillations produced by the encoding, but they cannot change their frequencies, which are fixed by how the data are encoded.

The observation also explains why practical QNNs rarely use a single rotation axis. Rotations about different axes do not commute and therefore do not reduce to a shift of the input. A sequence of rotations about two different axes, such as $R_z(\alpha)R_y(\beta)R_z(\gamma)$, can realize any single-qubit unitary up to a global phase [17, Sec. 4.2], which is why trainable layers typically combine R_x, R_y, and R_z gates, as in Figure 5. We retain the single-axis case here because it keeps the calculations transparent.

The three-qubit state. The same computation applies to each qubit. Defining the combined half-angles

$$\alpha_j = \frac{x_j + \theta_j}{2}, \qquad j = 1, 2, 3, \tag{3.37}$$

the three qubits are in the states

$$|q_0\rangle = \begin{bmatrix} \cos\alpha_1 \\ -i\sin\alpha_1 \end{bmatrix}, \qquad |q_1\rangle = \begin{bmatrix} \cos\alpha_2 \\ -i\sin\alpha_2 \end{bmatrix}, \qquad |q_2\rangle = \begin{bmatrix} \cos\alpha_3 \\ -i\sin\alpha_3 \end{bmatrix}. \tag{3.38}$$

Since every gate applied so far acts on a single qubit, the register is still in a product state. Following the little-endian convention, the full state vector is

$$|q_2 q_1 q_0\rangle = \begin{bmatrix} \cos\alpha_3 \\ -i\sin\alpha_3 \end{bmatrix} \otimes \begin{bmatrix} \cos\alpha_2 \\ -i\sin\alpha_2 \end{bmatrix} \otimes \begin{bmatrix} \cos\alpha_1 \\ -i\sin\alpha_1 \end{bmatrix} = \begin{bmatrix} \cos\alpha_3\cos\alpha_2\cos\alpha_1 & \leftarrow |000\rangle \\ -i\cos\alpha_3\cos\alpha_2\sin\alpha_1 & \leftarrow |001\rangle \\ -i\cos\alpha_3\sin\alpha_2\cos\alpha_1 & \leftarrow |010\rangle \\ -\cos\alpha_3\sin\alpha_2\sin\alpha_1 & \leftarrow |011\rangle \\ -i\sin\alpha_3\cos\alpha_2\cos\alpha_1 & \leftarrow |100\rangle \\ -\sin\alpha_3\cos\alpha_2\sin\alpha_1 & \leftarrow |101\rangle \\ -\sin\alpha_3\sin\alpha_2\cos\alpha_1 & \leftarrow |110\rangle \\ i\sin\alpha_3\sin\alpha_2\sin\alpha_1 & \leftarrow |111\rangle \end{bmatrix} \tag{3.39}$$

This is exactly the encoded state of Eq. (3.27) with each x_j replaced by $x_j + \theta_j$, as expected from Eq. (3.35).

The layer unitary. As for the encoding layer, the operator of the hidden layer on the full register is the tensor product of the single-qubit gates,

$$U_{\text{hidden}}(\boldsymbol{\theta}) = R_x(\theta_3) \otimes R_x(\theta_2) \otimes R_x(\theta_1), \tag{3.40}$$

an 8×8 matrix with exactly the structure of Eq. (3.29), with $c_j = \cos(x_j/2)$ and $s_j = \sin(x_j/2)$ replaced by $\cos(\theta_j/2)$ and $\sin(\theta_j/2)$; its entries are given by Eq. (3.30) with the same substitution. Because the tensor product satisfies $(A\otimes B)(C\otimes D) = AC\otimes BD$, the encoding and hidden layers together act as

$$U_{\text{hidden}}(\boldsymbol{\theta})\, U_{\text{enc}}(\mathbf{x}) = R_x(\theta_3 + x_3) \otimes R_x(\theta_2 + x_2) \otimes R_x(\theta_1 + x_1), \tag{3.41}$$

whose first column is Eq. (3.39).

The limitation of local rotations. Because the state remains a product state, the measurement statistics of each qubit depend only on its own input and its own parameter. For example, the probability of measuring q_0 in state $|0\rangle$ is $\cos^2\alpha_1$, regardless of x_2, x_3, θ_2, and θ_3. Adding further layers of single-qubit rotations cannot change this: no information flows between the qubits. In a classical neural network, this would correspond to a model in which each output depends on only one input feature, with no interaction between features. Introducing such interactions is the role of the entangling layer, examined next.

3.5 Hidden Layer: Entanglement

Parameterized quantum circuits are typically built from two kinds of layers: local rotations and entangling gates [26]. As shown in Section 3.4, rotation layers alone leave the register in a product state, so each qubit evolves independently and its measurement statistics depend only on its own input and parameters. The entangling layer removes this limitation. By applying CNOT gates between qubits, it allows the state of one qubit to influence another, so that information from different input features can be combined. This plays a role analogous to that of a fully connected layer in a classical neural network, in which every neuron receives input from all neurons in the preceding layer. Unlike a classical layer, however, the CNOT gate has no trainable parameters; its effect, as we show below, is to rearrange the amplitudes of the state vector.

Throughout this subsection we use the half-angles $\alpha_j = (x_j + \theta_j)/2$ introduced in Section 3.4.

Two qubits. We begin with the first two qubits, q_0 and q_1, as shown in Figure 8. After the encoding layer and the first rotation layer, they are in the states

$$|q_0\rangle = \cos\alpha_1\,|0\rangle - i\sin\alpha_1\,|1\rangle\,, \qquad |q_1\rangle = \cos\alpha_2\,|0\rangle - i\sin\alpha_2\,|1\rangle\,. \tag{3.42}$$

Following the little-endian convention, the two-qubit state is $|q_1q_0\rangle = |q_1\rangle \otimes |q_0\rangle$:

$$|q_1q_0\rangle = \begin{bmatrix}\cos\alpha_2\\ -i\sin\alpha_2\end{bmatrix} \otimes \begin{bmatrix}\cos\alpha_1\\ -i\sin\alpha_1\end{bmatrix} = \begin{bmatrix}\cos\alpha_2\cos\alpha_1 & \leftarrow |00\rangle\\ -i\cos\alpha_2\sin\alpha_1 & \leftarrow |01\rangle\\ -i\sin\alpha_2\cos\alpha_1 & \leftarrow |10\rangle\\ -\sin\alpha_2\sin\alpha_1 & \leftarrow |11\rangle\end{bmatrix} \tag{3.43}$$

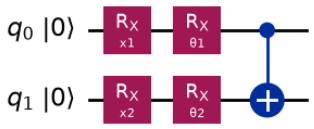


Figure 8: A CNOT gate with control q_0 and target q_1, applied after the encoding and rotation layers.

We now apply a CNOT gate with q_0 as the control and q_1 as the target, using the matrix of Eq. (3.12):

$$\mathrm{CNOT}_{q_0\to q_1}\,|q_1q_0\rangle = \begin{bmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{bmatrix}\begin{bmatrix}\cos\alpha_2\cos\alpha_1\\-i\cos\alpha_2\sin\alpha_1\\-i\sin\alpha_2\cos\alpha_1\\-\sin\alpha_2\sin\alpha_1\end{bmatrix} = \begin{bmatrix}\cos\alpha_2\cos\alpha_1\\-\sin\alpha_2\sin\alpha_1\\-i\sin\alpha_2\cos\alpha_1\\-i\cos\alpha_2\sin\alpha_1\end{bmatrix}. \tag{3.44}$$

The CNOT gate exchanges the second and fourth amplitudes, which belong to $|01\rangle$ and $|11\rangle$, the two basis states in which the control q_0 is in $|1\rangle$. No amplitude is created or modified; amplitudes are only reassigned to different basis states.

Verifying entanglement. Although the CNOT gate merely permutes amplitudes, the resulting state is generally entangled. To verify this, we apply the separability criterion of Eq. (3.18): a two-qubit state $a\,|00\rangle + b\,|01\rangle + c\,|10\rangle + d\,|11\rangle$ is a product state if and only if $ad - bc = 0$. Reading the coefficients from Eq. (3.44),

$$\begin{aligned} ad - bc &= (\cos\alpha_2\cos\alpha_1)\,(-i\cos\alpha_2\sin\alpha_1) - (-\sin\alpha_2\sin\alpha_1)\,(-i\sin\alpha_2\cos\alpha_1) \\ &= -i\cos\alpha_1\sin\alpha_1\left(\cos^2\alpha_2 + \sin^2\alpha_2\right) = -\frac{i}{2}\sin(2\alpha_1) = -\frac{i}{2}\sin(x_1+\theta_1). \end{aligned} \tag{3.45}$$

For comparison, the same quantity vanishes for the product state of Eq. (3.43), as it must. The state after the CNOT gate is therefore entangled whenever $\sin(x_1 + \theta_1) \neq 0$, that is,

$$\mathrm{CNOT}_{q_0\to q_1}\,|q_1q_0\rangle \neq |q_1'\rangle \otimes |q_0'\rangle \tag{3.46}$$

for any single-qubit states $|q_0'\rangle$ and $|q_1'\rangle$. Interestingly, the result depends only on the state of the control qubit. If the control is in a definite basis state, $|0\rangle$ or $|1\rangle$ (here, when $x_1 + \theta_1$ is a multiple of π), the CNOT gate either does nothing or flips the target, and no entanglement is created. Entanglement arises precisely when the control is in a superposition, so that the target is flipped in one branch of the superposition and not in the other.

Three qubits: the first CNOT. We now return to the full three-qubit register, starting from the product state of Eq. (3.39). The entangling layer of Figure 9 applies two CNOT gates in sequence: first $\mathrm{CNOT}_{q_0\to q_1}$, then $\mathrm{CNOT}_{q_1\to q_2}$.

The first CNOT acts on q_0 and q_1 and leaves q_2 unchanged. Its operator on the full register is therefore the tensor product of the identity on q_2 with the two-qubit CNOT matrix,

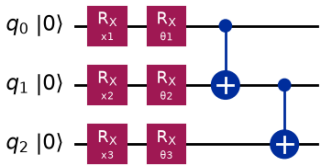


Figure 9: Entangling layer of the three-qubit QNN: $\mathrm{CNOT}_{q_0 \to q_1}$ followed by $\mathrm{CNOT}_{q_1 \to q_2}$.

with the factor for q_2 on the left, as required by the little-endian convention:

$$U_1 = I \otimes \mathrm{CNOT}_{q_0 \to q_1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}, \tag{3.47}$$

where I is the 2×2 identity. The matrix is block diagonal, with one copy of the two-qubit CNOT for each state of q_2. Applying it to Eq. (3.39) gives

$$U_1 \, |q_2 q_1 q_0\rangle = \begin{bmatrix} \cos\alpha_3 \cos\alpha_2 \cos\alpha_1 \\ -\cos\alpha_3 \sin\alpha_2 \sin\alpha_1 \\ -i\cos\alpha_3 \sin\alpha_2 \cos\alpha_1 \\ -i\cos\alpha_3 \cos\alpha_2 \sin\alpha_1 \\ -i\sin\alpha_3 \cos\alpha_2 \cos\alpha_1 \\ i\sin\alpha_3 \sin\alpha_2 \sin\alpha_1 \\ -\sin\alpha_3 \sin\alpha_2 \cos\alpha_1 \\ -\sin\alpha_3 \cos\alpha_2 \sin\alpha_1 \end{bmatrix} \begin{matrix} \leftarrow |000\rangle \\ \leftarrow |001\rangle \\ \leftarrow |010\rangle \\ \leftarrow |011\rangle \\ \leftarrow |100\rangle \\ \leftarrow |101\rangle \\ \leftarrow |110\rangle \\ \leftarrow |111\rangle \end{matrix} \tag{3.48}$$

Compared with Eq. (3.39), the amplitudes in positions $(1, 2, 3, 4, 5, 6, 7, 8)$ have been reordered as $(1, 4, 3, 2, 5, 8, 7, 6)$. Within each block of four amplitudes, which corresponds to a fixed state of q_2, the second and fourth amplitudes are exchanged, exactly as in the two-qubit case. These are the states $|\cdot 01\rangle$ and $|\cdot 11\rangle$, in which the control q_0 is in $|1\rangle$.

Three qubits: the second CNOT. The second gate uses q_1 as the control and q_2 as the target, and leaves q_0 unchanged. Its two-qubit matrix on $|q_2 q_1\rangle$ has the same form as before, since the control is again the less significant qubit, and the identity on q_0 now appears on the

right:

$$U_2 = \mathrm{CNOT}_{q_1 \to q_2} \otimes I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{3.49}$$

This gate flips q_2 whenever q_1 is in $|1\rangle$; it therefore exchanges $|010\rangle \leftrightarrow |110\rangle$ and $|011\rangle \leftrightarrow |111\rangle$, that is, positions $3 \leftrightarrow 7$ and $4 \leftrightarrow 8$. Applying it to Eq. (3.48) yields the output of the entangling layer:

$$U_2 U_1 \, |q_2 q_1 q_0\rangle = \begin{bmatrix} \cos\alpha_3 \cos\alpha_2 \cos\alpha_1 & \leftarrow |000\rangle \\ -\cos\alpha_3 \sin\alpha_2 \sin\alpha_1 & \leftarrow |001\rangle \\ -\sin\alpha_3 \sin\alpha_2 \cos\alpha_1 & \leftarrow |010\rangle \\ -\sin\alpha_3 \cos\alpha_2 \sin\alpha_1 & \leftarrow |011\rangle \\ -i\sin\alpha_3 \cos\alpha_2 \cos\alpha_1 & \leftarrow |100\rangle \\ i\sin\alpha_3 \sin\alpha_2 \sin\alpha_1 & \leftarrow |101\rangle \\ -i\cos\alpha_3 \sin\alpha_2 \cos\alpha_1 & \leftarrow |110\rangle \\ -i\cos\alpha_3 \cos\alpha_2 \sin\alpha_1 & \leftarrow |111\rangle \end{bmatrix} \tag{3.50}$$

The full entangling layer is $U_{\mathrm{ent}} = U_2 U_1$; the gate applied first appears on the right.

CNOT gates as permutations. These calculations illustrate a general property. A CNOT gate maps each computational basis state to another basis state, flipping the target bit whenever the control bit is 1, so its matrix is a *permutation matrix*. The same reasoning applies to a CNOT between any pair of qubits, such as q_0 and q_2. Consequently, the entangling layer introduces no new trigonometric functions: every amplitude in Eq. (3.50) is still a product of $\cos\alpha_j$ and $\sin\alpha_j$ terms, exactly as before the layer. What changes is *which* product is attached to *which* basis state. This reassignment is enough to destroy the product structure. For generic inputs and parameters, the state of Eq. (3.50) is entangled,

$$U_{\mathrm{ent}} \, |q_2 q_1 q_0\rangle \neq |q_2'\rangle \otimes |q_1'\rangle \otimes |q_0'\rangle \,, \tag{3.51}$$

so the qubits can no longer be described individually. As Section 3.6 shows, this reassignment is what allows the measurement statistics of one qubit to depend on the inputs and parameters of the others.

3.6 Information Propagation through CNOT Gates

Section 3.5 showed that the entangling layer rearranges the amplitudes of the state vector. We now examine what this rearrangement means for the quantity that is actually observed: the probability of each measurement outcome. We show that a CNOT gate transfers information from its control qubit to its target qubit, so that the measurement statistics of the target

come to depend on the inputs and parameters of the control. Throughout, measurements are performed in the computational (Z) basis, and we continue to use the half-angles $\alpha_j = (x_j + \theta_j)/2$.

Two qubits before the CNOT gate. The probability of measuring q_1 in state $|0\rangle$ is the sum of the squared magnitudes of all amplitudes whose q_1 bit is 0, namely those of $|00\rangle$ and $|01\rangle$. From the product state of Eq. (3.43),

$$\begin{aligned} P(q_1 = 0) &= |\cos\alpha_2 \cos\alpha_1|^2 + |-i\cos\alpha_2 \sin\alpha_1|^2 \\ &= \cos^2\alpha_2 \left(\cos^2\alpha_1 + \sin^2\alpha_1\right) = \cos^2\left(\frac{x_2+\theta_2}{2}\right). \end{aligned} \tag{3.52}$$

The input x_1 and parameter θ_1 of the other qubit cancel out: the outcome of q_1 depends only on its own input and parameter, $P(q_1 = 0) = f(x_2, \theta_2)$.

Two qubits after the CNOT gate. After $\mathrm{CNOT}_{q_0 \to q_1}$, the state is given by Eq. (3.44), in which the amplitudes of $|00\rangle$ and $|01\rangle$ are $\cos\alpha_2\cos\alpha_1$ and $-\sin\alpha_2\sin\alpha_1$. Hence

$$\begin{aligned} P(q_1 = 0) &= \cos^2\alpha_2\cos^2\alpha_1 + \sin^2\alpha_2\sin^2\alpha_1 \\ &= \frac{1}{2}\left[1 + \cos(2\alpha_1)\cos(2\alpha_2)\right] = \frac{1}{2}\left[1 + \cos(x_1+\theta_1)\cos(x_2+\theta_2)\right], \end{aligned} \tag{3.53}$$

where the second line follows from $\cos^2 a = \frac{1}{2}(1+\cos 2a)$ and $\sin^2 a = \frac{1}{2}(1-\cos 2a)$. The outcome of q_1 now depends on all four quantities, $P(q_1 = 0) = f(x_1, x_2, \theta_1, \theta_2)$: the input and parameter of q_0 have been transferred to q_1. This is the mechanism by which the entangling layer allows a QNN to combine information from different features.

The measurement statistics of the control qubit, by contrast, are unchanged:

$$P(q_0 = 0) = \cos^2\left(\frac{x_1+\theta_1}{2}\right) \tag{3.54}$$

both before and after the gate. This is a direct consequence of the CNOT's action on basis states, $|c\rangle|t\rangle \mapsto |c\rangle|t \oplus c\rangle$: the control bit is never modified, so the total probability of each control value is preserved. In the computational basis, information therefore flows only from the control to the target. This asymmetry is specific to Z-basis measurements. In other bases, the CNOT gate can affect the control qubit as well, an effect known as phase kickback [17].

Comparison with classical neural networks. In a fully connected classical layer, each neuron computes a function of all its inputs,

$$f_{\mathrm{NN}}(\mathbf{x}) = f\Big(\sigma\Big(\mathbf{w}^T\mathbf{x} + b\Big)\Big) = f\left(\sigma\left(\sum_{n=1}^{N} w_n x_n + b\right)\right), \tag{3.55}$$

so that every output depends on $x_1, x_2, \ldots, x_N$ jointly. A network whose outputs each depended on a single input feature could not capture interactions between features and would be severely limited in expressivity. Without entangling gates, a QNN is in exactly this situation, as Eq. (3.52) shows. The entangling layer plays the role of the connections between neurons, with the difference that it introduces no trainable parameters.

Three qubits. The same analysis applies to the three-qubit circuit. Before the entangling layer, the state is the product state of Eq. (3.39), and the probability of measuring q_2 in $|0\rangle$ is the sum over the four basis states $|0b_1b_0\rangle$:

$$\begin{aligned} P(q_2=0) &= |\psi_{000}|^2 + |\psi_{001}|^2 + |\psi_{010}|^2 + |\psi_{011}|^2 \\ &= \cos^2\alpha_3 \left[\cos^2\alpha_2 \left(\cos^2\alpha_1 + \sin^2\alpha_1\right) + \sin^2\alpha_2 \left(\cos^2\alpha_1 + \sin^2\alpha_1\right)\right] \\ &= \cos^2\alpha_3 = \cos^2\left(\frac{x_3+\theta_3}{2}\right). \end{aligned} \tag{3.56}$$

As in the two-qubit case, the outcome depends only on the qubit's own input and parameter. This remains true however many single-qubit rotation layers are added, since such layers never mix the qubits.

After the entangling layer, the state is given by Eq. (3.50). Summing the squared magnitudes of its first four amplitudes and grouping terms gives

$$\begin{aligned} P(q_2=0) &= \cos^2\alpha_3 \left[\cos^2\alpha_2 \cos^2\alpha_1 + \sin^2\alpha_2 \sin^2\alpha_1\right] \\ &\quad + \sin^2\alpha_3 \left[\sin^2\alpha_2 \cos^2\alpha_1 + \cos^2\alpha_2 \sin^2\alpha_1\right] \\ &= \frac{\cos^2\alpha_3}{2}\left[1 + \cos(2\alpha_2)\cos(2\alpha_1)\right] + \frac{\sin^2\alpha_3}{2}\left[1 - \cos(2\alpha_2)\cos(2\alpha_1)\right] \\ &= \frac{1}{2}\left[1 + \cos(2\alpha_3)\cos(2\alpha_2)\cos(2\alpha_1)\right] = \frac{1}{2}\left[1 + \prod_{j=1}^{3}\cos(x_j+\theta_j)\right], \end{aligned} \tag{3.57}$$

where the second line uses the same identities as Eq. (3.53), and the third uses $\cos^2\alpha_3 - \sin^2\alpha_3 = \cos(2\alpha_3)$. Comparing Eqs. (3.56) and (3.57),

$$P_{\text{before}}(q_2=0) = \cos^2\left(\frac{x_3+\theta_3}{2}\right), \qquad P_{\text{after}}(q_2=0) = \frac{1}{2}\left[1 + \prod_{j=1}^{3}\cos(x_j+\theta_j)\right]. \tag{3.58}$$

Equivalently, since $\langle Z\rangle = P(0) - P(1) = 2P(0) - 1$ (Section 3.7), the expectation value of Z on q_2 changes from $\cos(x_3+\theta_3)$ to the product $\cos(x_1+\theta_1)\cos(x_2+\theta_2)\cos(x_3+\theta_3)$. After the entangling layer, the outcome of q_2 depends on every input and every parameter. The information has propagated along the chain of CNOT gates: from q_0 to q_1 through the first gate, and from q_1 to q_2 through the second. Because q_0 is never a target, its statistics remain $\cos^2\alpha_1$; a ring of CNOT gates, with an additional gate from q_2 back to q_0, would close the loop.

The cost of classical simulation. Once the qubits are entangled, the state can no longer be described qubit by qubit; it must be tracked as a vector of 2^n complex amplitudes. Storing such a vector in double precision requires $16 \cdot 2^n$ bytes, which amounts to 16 GiB for $n = 30$ qubits, 16 TiB for $n = 40$, and 16 PiB for $n = 50$. Every additional qubit doubles the memory and the cost of applying each gate, so exact state-vector simulation becomes infeasible beyond roughly fifty qubits, whereas current quantum processors already operate with more than one hundred qubits [20]. This exponential growth is the underlying reason why quantum circuits

may be hard to simulate classically. It does not by itself imply a computational advantage for QNNs, however: information can be extracted from the state only through repeated measurements (Section 3.7), many circuits of practical interest can be simulated efficiently by other classical methods, and, as discussed in Section 5.7, the functions computed by QNNs can often be approximated classically.

A second hidden layer. As Figure 10 shows, the circuit continues with a second rotation layer after the entangling layer, with trainable angles θ_4, θ_5, and θ_6 on q_0, q_1, and q_2, respectively.

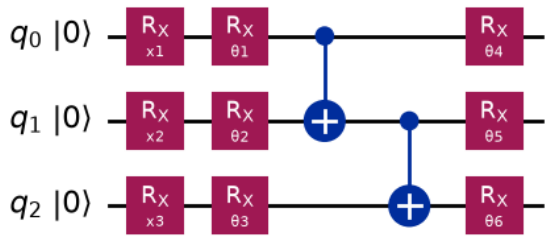


Figure 10: The three-qubit QNN with two hidden rotation layers separated by an entangling layer.

The operator of this layer is

$$U^{(2)}_{\text{hidden}} = R_x(\theta_6) \otimes R_x(\theta_5) \otimes R_x(\theta_4), \tag{3.59}$$

an 8×8 matrix with the structure of Eq. (3.29), with c_j and s_j now equal to $\cos(\theta_j/2)$ and $\sin(\theta_j/2)$ for $j = 4, 5, 6$. It acts on the entangled state of Eq. (3.50):

$$|\psi\rangle = U^{(2)}_{\text{hidden}}\, U_{\text{ent}}\, U_{\text{hidden}}(\boldsymbol{\theta})\, U_{\text{enc}}(\mathbf{x})\, |000\rangle\,. \tag{3.60}$$

The calculation simplifies for q_2. A rotation R_x on the target of a CNOT commutes with the CNOT, because the CNOT applies either I or X to the target, and both commute with R_x. Since q_2 is the target of $\text{CNOT}_{q_1 \to q_2}$ and is not involved in $\text{CNOT}_{q_0 \to q_1}$, the gate $R_x(\theta_6)$ can be moved through the entire entangling layer and merged with the first rotation on q_2, giving a single rotation by $x_3 + \theta_3 + \theta_6$. No such simplification is possible for q_0 and q_1, which act as controls. Defining

$$\begin{aligned} C_j &= \cos\alpha_j, \quad S_j = \sin\alpha_j \quad (j = 1, 2), \\ C_3 &= \cos\left(\frac{x_3 + \theta_3 + \theta_6}{2}\right), \quad S_3 = \sin\left(\frac{x_3 + \theta_3 + \theta_6}{2}\right), \\ c_j &= \cos\left(\frac{\theta_j}{2}\right), \quad s_j = \sin\left(\frac{\theta_j}{2}\right) \quad (j = 4, 5), \end{aligned} \tag{3.61}$$

the resulting state is

$$|\psi\rangle = \begin{bmatrix} C_3c_5(c_4C_2C_1 + is_4S_2S_1) + S_3s_5(ic_4S_2C_1 + s_4C_2S_1) \\ C_3c_5(-is_4C_2C_1 - c_4S_2S_1) + S_3s_5(s_4S_2C_1 + ic_4C_2S_1) \\ C_3s_5(-ic_4C_2C_1 + s_4S_2S_1) + S_3c_5(-c_4S_2C_1 + is_4C_2S_1) \\ C_3s_5(-s_4C_2C_1 + ic_4S_2S_1) + S_3c_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3c_5(-ic_4C_2C_1 + s_4S_2S_1) + C_3s_5(-c_4S_2C_1 + is_4C_2S_1) \\ S_3c_5(-s_4C_2C_1 + ic_4S_2S_1) + C_3s_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3s_5(-c_4C_2C_1 - is_4S_2S_1) + C_3c_5(-ic_4S_2C_1 - s_4C_2S_1) \\ S_3s_5(is_4C_2C_1 + c_4S_2S_1) + C_3c_5(-s_4S_2C_1 - ic_4C_2S_1) \end{bmatrix}. \tag{3.62}$$

From state vectors to Fourier series. Even for three qubits and two hidden layers, the explicit state vector is already unwieldy, and deriving it by hand quickly becomes intractable as the circuit grows. A more compact description is needed to analyze QNNs systematically. Fortunately, the expressions above have a simple underlying structure that leads directly to one.

Each input x_j enters the circuit through exactly one gate, the encoding rotation $R_x(x_j)$, and every other gate is independent of the inputs. By Eq. (3.30), every entry of the encoding unitary is a product containing exactly one factor $\cos(x_j/2)$ or $\sin(x_j/2)$ for each input. Since the remaining gates only form linear combinations of these entries, every amplitude of the final state, including those in Eq. (3.62), is a linear combination of such products, with coefficients determined by the trainable parameters. Writing $\cos(x_j/2)$ and $\sin(x_j/2)$ in terms of $e^{\pm ix_j/2}$ shows that each amplitude contains only the half-integer frequencies $\pm\frac{1}{2}$ in each input. The measured output is quadratic in the amplitudes, since it involves both ψ and ψ^*. In the output, the half-integer frequencies therefore combine pairwise into frequencies $\omega_j \in \{-1, 0, 1\}$ for each input, and the model takes the form

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{\boldsymbol{\omega}\in\{-1,0,1\}^3} c_{\boldsymbol{\omega}}(\boldsymbol{\theta})\, e^{i\boldsymbol{\omega}\cdot\mathbf{x}}. \tag{3.63}$$

Equation (3.57) illustrates this structure concretely: expanding the product $\prod_j \cos(x_j + \theta_j)$ by $\cos u = \frac{1}{2}(e^{iu} + e^{-iu})$ yields exactly such a sum, with frequency vectors $(\pm1, \pm1, \pm1)$ and coefficients whose phases are set by the parameters.

Equation (3.63) is a special case of a general result: under the encoding schemes considered in this paper, the output of a QNN is a finite Fourier series [35]. Its frequencies $\boldsymbol{\omega}$ are fixed by the encoding, while only the coefficients $c_{\boldsymbol{\omega}}(\boldsymbol{\theta})$ are trainable. This contrasts with classical networks such as SIREN, whose frequencies can be adapted during training [38]. The connection allows the expressive power of a QNN to be studied through its frequency spectrum and Fourier coefficients, rather than through its exponentially large state vector. Section 4 develops this framework in detail: it derives the Fourier representation and shows how the number of encoding qubits and re-uploading layers determines the accessible frequencies. Section 5.3 then addresses the question raised in the Introduction of whether a predetermined spectrum is a limitation or an advantage.

3.7 Output Layer: Quantum Measurement

The final stage of a QNN is the measurement layer, which converts the quantum state into classical numbers. This stage has no direct counterpart in classical neural networks, where every intermediate activation can be read directly from memory. The amplitudes of a quantum state, by contrast, cannot be observed directly. A measurement of a qubit in the computational basis returns a single bit, 0 or 1, with probabilities given by the Born rule [17, Sec. 2.2], and the measurement disturbs the state. A single run of the circuit therefore reveals neither the complex amplitudes nor even the probabilities themselves. This subsection describes how a QNN nevertheless produces well-defined real-valued outputs: by estimating the *expectation value* of an observable from repeated runs.

The QNN output as an expectation value. After the encoding layer and the trainable rotation and entangling layers, the state of the n-qubit register is

$$|\psi(\mathbf{x}, \boldsymbol{\theta})\rangle = U(\mathbf{x}, \boldsymbol{\theta}) \, |0\rangle^{\otimes n} \, , \tag{3.64}$$

where $U(\mathbf{x}, \boldsymbol{\theta})$ is the product of all gates in the circuit. However many layers the circuit contains, the ideal circuit as a whole is described by this single unitary operator; for the three-qubit example of the previous subsections, $U = U^{(2)}_{\text{hidden}} U_{\text{ent}} U_{\text{hidden}} U_{\text{enc}}$. Given a Hermitian operator O, called an *observable*, the output of the QNN is defined as its expectation value in this state:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \langle \psi(\mathbf{x}, \boldsymbol{\theta})| \; O \; |\psi(\mathbf{x}, \boldsymbol{\theta})\rangle = \langle 0|^{\otimes n} \; U^{\dagger}(\mathbf{x}, \boldsymbol{\theta}) \, O \, U(\mathbf{x}, \boldsymbol{\theta}) \; |0\rangle^{\otimes n} \, . \tag{3.65}$$

Because O is Hermitian, it has real eigenvalues λ_k and can be written as $O = \sum_k \lambda_k \Pi_k$, where Π_k projects onto the eigenspace of λ_k. Measuring O yields the outcome λ_k with probability $P(\lambda_k) = \langle \psi| \, \Pi_k \, |\psi\rangle$, and the expectation value is the probability-weighted average of the outcomes,

$$\langle O \rangle = \sum_k \lambda_k P(\lambda_k). \tag{3.66}$$

The output $f_{\boldsymbol{\theta}}(\mathbf{x})$ is therefore always a real number, even though the amplitudes of the state are complex.

The Pauli-Z observable. The most commonly used observable in variational QNNs is the Pauli-Z operator,

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = (+1) \, |0\rangle \langle 0| + (-1) \, |1\rangle \langle 1| \, , \tag{3.67}$$

whose eigenstates are the computational basis states: $|0\rangle$ with eigenvalue $+1$ and $|1\rangle$ with eigenvalue -1. Measuring Z is therefore the same as measuring the qubit in the computational basis and recording $+1$ for outcome 0 and -1 for outcome 1.

For a single qubit in the state $|\psi\rangle = \alpha \, |0\rangle + \beta \, |1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$, the Born rule gives the outcome probabilities $P(0) = |\alpha|^2$ and $P(1) = |\beta|^2$. The expectation value of Z is

$$\langle Z \rangle = \langle \psi| \, Z \, |\psi\rangle = \begin{bmatrix} \alpha^* & \beta^* \end{bmatrix} \begin{bmatrix} \alpha \\ -\beta \end{bmatrix} = |\alpha|^2 - |\beta|^2 = P(0) - P(1) = 2P(0) - 1, \tag{3.68}$$

where the last step uses $P(0) + P(1) = 1$. Conversely, the probabilities can be recovered from the expectation value:

$$P(0) = \frac{1 + \langle Z \rangle}{2}, \qquad P(1) = \frac{1 - \langle Z \rangle}{2}. \tag{3.69}$$

Since $-1 \leq \langle Z \rangle \leq 1$, a value close to $+1$ indicates that the qubit is most likely to be found in $|0\rangle$, and a value close to -1 that it is most likely to be found in $|1\rangle$. For the encoded state of Eq. (3.23), for example, $\langle Z \rangle = \cos^2(x_1/2) - \sin^2(x_1/2) = \cos x_1$.

The expectation value should not be confused with a probability amplitude. The amplitudes α and β are complex numbers that are not directly observable, whereas $\langle Z \rangle$ is a real number that depends only on the magnitudes $|\alpha|$ and $|\beta|$. In particular, the relative phase between α and β does not affect $\langle Z \rangle$; it influences the output only through the gates that act before the measurement.

Estimating expectation values on hardware. Each execution of the circuit, called a *shot*, yields a single outcome, and the measurement destroys the superposition. The expectation value must therefore be estimated statistically by preparing and measuring the state repeatedly [25, 26]. If N_0 and N_1 denote the numbers of outcomes 0 and 1 in $N = N_0 + N_1$ shots, the estimator is

$$\widehat{\langle Z \rangle} = \frac{N_0 - N_1}{N}. \tag{3.70}$$

This estimator is the sample mean of N independent outcomes $z \in \{+1, -1\}$, each with mean $\langle Z \rangle$ and variance $\mathbb{E}[z^2] - \langle Z \rangle^2 = 1 - \langle Z \rangle^2$. Its standard error is therefore

$$\sigma\left(\widehat{\langle Z \rangle}\right) = \sqrt{\frac{1 - \langle Z \rangle^2}{N}} \leq \frac{1}{\sqrt{N}}. \tag{3.71}$$

Reaching a precision ϵ requires on the order of $1/\epsilon^2$ shots; for instance, a precision of 0.01 requires up to 10,000 shots. This sampling overhead applies to every input and every evaluation during training, and it is one of the main practical costs of QNNs compared with classical networks.

Multi-qubit measurements. The same principles apply to a register of several qubits. For the three-qubit QNN, the final state can be written as

$$|\psi\rangle = \sum_{b_2, b_1, b_0 \in \{0,1\}} a_{b_2 b_1 b_0} \, |b_2 b_1 b_0\rangle \,, \qquad \sum_{b_2, b_1, b_0} |a_{b_2 b_1 b_0}|^2 = 1, \tag{3.72}$$

where, under the little-endian convention, the amplitudes $a_{000}, a_{001}, \ldots, a_{111}$ are the entries of the state vector in order. To measure a single qubit, Z is applied to that qubit and the identity to all others. Since the factor acting on q_2 appears on the left, the observables for the three qubits are

$$Z_{q_0} = I \otimes I \otimes Z, \qquad Z_{q_1} = I \otimes Z \otimes I, \qquad Z_{q_2} = Z \otimes I \otimes I. \tag{3.73}$$

Each of these operators is diagonal, with entry $(-1)^{b_k}$ for the basis state $|b_2 b_1 b_0\rangle$, where b_k is the bit of the measured qubit. For q_0, for example,

$$\begin{aligned}\langle Z_{q_0}\rangle = \langle\psi| Z_{q_0} |\psi\rangle = |a_{000}|^2 - |a_{001}|^2 + |a_{010}|^2 - |a_{011}|^2 \\ + |a_{100}|^2 - |a_{101}|^2 + |a_{110}|^2 - |a_{111}|^2 = P(q_0 = 0) - P(q_0 = 1).\end{aligned} \tag{3.74}$$

Measuring a single qubit thus involves all basis states: the result depends on the *marginal* probabilities of that qubit, obtained by summing over the states of all other qubits, rather than on any individual amplitude. This is precisely the calculation carried out in Section 3.6. There, for instance, the entangling layer turned $\langle Z_{q_2}\rangle$ from $\cos(x_3+\theta_3)$ into $\cos(x_1+\theta_1)\cos(x_2+\theta_2)\cos(x_3+\theta_3)$.

Measuring all three qubits yields a vector of expectation values,

$$\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}) = \begin{bmatrix}\langle Z_{q_0}\rangle \\ \langle Z_{q_1}\rangle \\ \langle Z_{q_2}\rangle\end{bmatrix}, \tag{3.75}$$

which serves as the classical output of the QNN. It can be used directly for prediction, for example by thresholding a single component for binary classification, or passed to a subsequent classical layer. Since the three observables are all diagonal in the computational basis, they can be estimated from the same set of shots: each shot measures all qubits simultaneously, and each component is computed from the corresponding bit.

Other observables. Observables other than single-qubit Z operators can extract different information from the state. Products such as $Z_{q_0} Z_{q_1}$ measure correlations between qubits; these are also diagonal in the computational basis and can be estimated from the same shots. Pauli-X and Pauli-Y observables are measured by applying a basis-changing gate (a Hadamard gate for X) before a computational-basis measurement. A detailed treatment of measurement strategies is beyond the scope of this paper.

Summary. The measurement layer bridges the quantum state and the classical output of a QNN. Expressing this output as the expectation value of Eq. (3.65) is also the starting point for the analysis in Section 4.2. Because the expectation value is quadratic in the amplitudes, involving both U and $U^\dagger$, frequencies from the state vector combine pairwise in the output, which gives rise to the finite Fourier series structure of QNNs.

4 Fourier Analysis of Quantum Neural Networks and Expressivity

4.1 Basics of Fourier Series and Fourier Transform

Fourier analysis provides a powerful mathematical framework for representing and analyzing functions in terms of their frequency components. Standard treatments cover both Fourier series and Fourier transforms [44]. The fundamental idea is that a function can be decomposed into a combination of oscillatory basis functions, such as sines, cosines, or complex exponentials.

Instead of examining a function solely in its original domain, Fourier analysis allows us to study its behavior in the frequency domain.

Two fundamental tools in Fourier analysis are the *Fourier series* and the *Fourier transform*. Although closely related, they serve different purposes and apply to different classes of functions.

Fourier Series

A Fourier series represents a periodic function as an infinite sum of sinusoidal components with different frequencies, amplitudes, and phases.

For a periodic function $f(x)$ with period T, its Fourier series can be expressed in complex exponential form as

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ik\omega_0 x}, \tag{4.1}$$

where $\omega_0 = 2\pi/T$ is the fundamental angular frequency, $k \in \mathbb{Z}$ is the harmonic index, and c_k denotes the Fourier coefficient associated with frequency $k\omega_0$.

Each Fourier coefficient can be calculated by projecting the original function onto the corresponding complex exponential basis function:

$$c_k = \frac{1}{T} \int_0^T f(x) e^{-ik\omega_0 x}\, dx. \tag{4.2}$$

These coefficients describe the contribution of each frequency component to the original function. In particular, $|c_k|$ determines the amplitude of the corresponding component, while the argument of c_k determines its phase shift.

By Euler's formula,

$$e^{i\omega x} = \cos(\omega x) + i \sin(\omega x), \tag{4.3}$$

so the complex exponential representation is equivalent to expressing a real-valued periodic function as a sum of sine and cosine functions.

Importantly, different frequencies capture different levels of variation in a function. Low-frequency components describe slowly varying patterns, whereas high-frequency components represent rapid oscillations and fine-grained details.

In practical applications, Fourier series are often approximated using a finite number of frequency components. More generally, a finite trigonometric expansion can be written as

$$f(x) = \sum_{\omega \in \Omega} c_\omega e^{i\omega x}, \tag{4.4}$$

where Ω is a finite set of frequencies. Unlike in a conventional Fourier series, the frequencies in this generalized representation are not necessarily integer multiples of a common fundamental frequency.

Fourier Transform

While Fourier series decompose periodic functions into discrete harmonics, the Fourier transform extends frequency-domain analysis to nonperiodic functions.

For a sufficiently well-behaved function $f(x)$, the continuous Fourier transform is defined as

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x}\, dx, \tag{4.5}$$

where $\omega \in \mathbb{R}$ represents the angular frequency and $\hat{f}(\omega)$ describes the frequency-domain representation of $f(x)$.

The original function can be reconstructed using the inverse Fourier transform:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x} \, d\omega. \tag{4.6}$$

The key distinction is that the Fourier series uses a discrete set of frequencies, whereas the Fourier transform is defined over a continuous frequency variable. Nevertheless, both share the same fundamental principle: a function can be understood through its frequency components and their corresponding coefficients.

From a function-approximation perspective, the frequency spectrum reveals which oscillatory patterns are present in a function, and this is the perspective we adopt for QNNs. As previewed in Section 1.4 and in the three-qubit example of Section 3.6, the output of a QNN takes the form of the finite expansion in Eq. (4.4), with a frequency set Ω fixed by the data encoding and coefficients $c_\omega(\boldsymbol{\theta})$ controlled by the trainable parameters [35]. Unlike in conventional Fourier approximations, the frequencies are therefore not chosen or optimized for a particular task, and Fourier analysis provides a natural framework for two fundamental aspects of QNN expressivity: which frequencies a circuit can represent, and how flexibly it can adjust their coefficients. Section 4.2 derives this representation, and Section 4.3 examines how the encoding strategy determines the accessible spectrum.

4.2 Quantum Neural Networks as Finite Fourier Series

This subsection presents a detailed derivation of a result due to Schuld, Sweke, and Meyer [35]: the output of a quantum neural network (QNN) in which the input is encoded through Hamiltonian-generated gates can be expressed as a finite Fourier series in that input. The argument proceeds in four stages. First, we fix the general form of the circuit and show that the encoding gates may, without loss of generality, be taken to be diagonal. Second, we compute the state vector for one, two, and finally L encoding layers, and show that every amplitude is a sum of complex exponentials whose frequencies are sums of eigenvalues of the encoding Hamiltonian. Third, we evaluate the measured expectation value and show that the frequencies of the model output are *differences* of these eigenvalue sums. Finally, we collect terms of equal frequency to obtain the Fourier representation and illustrate it with Pauli-rotation encoding.

Conventions. We consider a scalar input $x \in \mathbb{R}$ and a circuit on N qubits, so that the Hilbert space has dimension $d = 2^N$. Computational basis states are indexed by $1, \ldots, d$, with the initial state $|0\rangle \equiv |0\rangle^{\otimes N}$ corresponding to the first basis vector. For a matrix W, W_{mn} denotes the entry in row m and column n. The symbol i is reserved for the imaginary unit; matrix and vector indices are denoted by m, n, j, and k.

Circuit model. The quantum circuits considered here share the general structure shown in Figure 11, in which data-independent blocks alternate with data-encoding gates. The overall unitary is therefore

$$U(x) = W^{(L+1)} S(x) \, W^{(L)} \cdots W^{(2)} S(x) \, W^{(1)}, \tag{4.7}$$

where each $W^{(l)}$ is a unitary that does not depend on x. These blocks may contain trainable parameterized gates, such as $R_x(\theta)$, as well as fixed gates, such as the Hadamard gate. The gate $S(x)$ encodes the classical input x into the quantum state and is repeated L times. We assume that $S(x)$ is generated by a Hermitian operator H, referred to as the *encoding Hamiltonian*:

$$S(x) = e^{-ixH}. \tag{4.8}$$

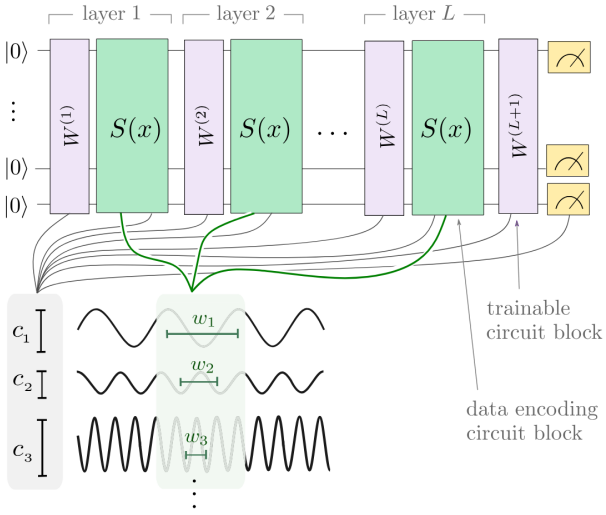


Figure 11: General structure of the quantum circuits considered in this section: data-independent blocks $W^{(l)}$ alternate with data-encoding gates $S(x)$.

For example, encoding with the single-qubit rotation $R_x(x)$ corresponds to

$$H = \frac{1}{2}X = \frac{1}{2}\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{4.9}$$

Indeed, since $X^2 = I$, the Taylor series of the exponential separates into even powers, which are proportional to I, and odd powers, which are proportional to X:

$$\begin{aligned} e^{-i\frac{x}{2}X} &= \sum_{n=0}^{\infty} \frac{1}{n!}\left(-\frac{ix}{2}\right)^n X^n \\ &= \left[\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!}\left(\frac{x}{2}\right)^{2k}\right] I - i\left[\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!}\left(\frac{x}{2}\right)^{2k+1}\right] X \\ &= \cos\left(\frac{x}{2}\right) I - i\sin\left(\frac{x}{2}\right) X = R_x(x). \end{aligned} \tag{4.10}$$

The output of the model is the expectation value of a Hermitian observable O (which need not be unitary) with respect to the final state:

$$f(x) = \langle\psi(x)|\, O\, |\psi(x)\rangle\,, \qquad |\psi(x)\rangle = U(x)\,|0\rangle\,. \tag{4.11}$$

The trainable parameters $\boldsymbol{\theta}$ enter only through the blocks $W^{(l)}$; for brevity, we suppress the dependence on θ throughout.

Step 1: Diagonalizing the encoding gate. Because H is Hermitian, the spectral theorem guarantees the decomposition

$$H = V^\dagger \Sigma V, \qquad \Sigma = \mathrm{diag}(\lambda_1, \ldots, \lambda_d), \tag{4.12}$$

where V is unitary and the eigenvalues $\lambda_1, \ldots, \lambda_d$ are real. Since V is unitary, $(V^\dagger \Sigma V)^n = V^\dagger \Sigma^n V$ for every n, and the power-series definition of the matrix exponential gives

$$S(x) = V^\dagger e^{-ix\Sigma} V, \qquad e^{-ix\Sigma} = \mathrm{diag}\Big(e^{-ix\lambda_1}, \ldots, e^{-ix\lambda_d}\Big). \tag{4.13}$$

Substituting this expression into Eq. (4.7), every factor V or $V^\dagger$ is adjacent to a data-independent block:

$$U(x) = W^{(L+1)} V^\dagger e^{-ix\Sigma} V \, W^{(L)} V^\dagger e^{-ix\Sigma} V \cdots V \, W^{(2)} V^\dagger e^{-ix\Sigma} V \, W^{(1)}. \tag{4.14}$$

These factors can therefore be absorbed into the neighboring blocks by defining

$$\widetilde{W}^{(1)} = V W^{(1)}, \qquad \widetilde{W}^{(l)} = V W^{(l)} V^\dagger \;\; (2 \le l \le L), \qquad \widetilde{W}^{(L+1)} = W^{(L+1)} V^\dagger, \tag{4.15}$$

which yields

$$U(x) = \widetilde{W}^{(L+1)} e^{-ix\Sigma} \, \widetilde{W}^{(L)} \cdots \widetilde{W}^{(2)} e^{-ix\Sigma} \, \widetilde{W}^{(1)}. \tag{4.16}$$

The new blocks $\widetilde{W}^{(l)}$ are still unitary and independent of x. Consequently, we may assume without loss of generality that the encoding gate is diagonal in the computational basis. To simplify notation, we drop the tildes and write from now on

$$S(x) = \mathrm{diag}\Big(e^{-ix\lambda_1}, \ldots, e^{-ix\lambda_d}\Big). \tag{4.17}$$

Step 2: A single encoding layer ($L = 1$). We now compute the amplitudes of the final state, beginning with the simplest case,

$$U(x) = W^{(2)} S(x) \, W^{(1)}. \tag{4.18}$$

Because $|0\rangle$ is the first basis vector, the first block maps it to the first column of $W^{(1)}$:

$$|\phi\rangle = W^{(1)} |0\rangle = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{bmatrix}, \qquad a_j = W^{(1)}_{j1}. \tag{4.19}$$

Since $S(x)$ is diagonal, the encoding gate multiplies each component by a phase:

$$S(x) \, |\phi\rangle = \begin{bmatrix} a_1 e^{-ix\lambda_1} \\ a_2 e^{-ix\lambda_2} \\ \vdots \\ a_d e^{-ix\lambda_d} \end{bmatrix}. \tag{4.20}$$

Applying the second block $W^{(2)}$, the m-th component of the final state is

$$\begin{aligned} \psi_m(x) &= W^{(2)}_{m1} a_1 e^{-ix\lambda_1} + W^{(2)}_{m2} a_2 e^{-ix\lambda_2} + \cdots + W^{(2)}_{md} a_d e^{-ix\lambda_d} \\ &= \sum_{j=1}^{d} W^{(2)}_{mj} W^{(1)}_{j1} \, e^{-ix\lambda_j}. \end{aligned} \tag{4.21}$$

Each amplitude is thus a linear combination of the exponentials $e^{-ix\lambda_j}$, with coefficients that depend only on the data-independent blocks.

Step 3: Two encoding layers ($L = 2$). Next, consider

$$U(x) = W^{(3)} S(x)\, W^{(2)} S(x)\, W^{(1)}. \tag{4.22}$$

Let $\phi^{(l)}$ denote the state immediately after block $W^{(l)}$, and $\hat{\phi}^{(l)}$ the state after the encoding gate that follows it. The indices $j_1, j_2 \in \{1, \ldots, d\}$ are independent. After the first block,

$$\phi_{j_1}^{(1)} = W_{j_1 1}^{(1)}, \tag{4.23}$$

and after the first encoding gate,

$$\hat{\phi}_{j_1}^{(1)} = e^{-ix\lambda_{j_1}} W_{j_1 1}^{(1)}. \tag{4.24}$$

Applying the second block gives

$$\phi_{j_2}^{(2)} = \sum_{j_1=1}^{d} W_{j_2 j_1}^{(2)} e^{-ix\lambda_{j_1}} W_{j_1 1}^{(1)}. \tag{4.25}$$

The sum arises because the j_2-th row of $W^{(2)}$ combines all d components of the preceding state. The second encoding gate multiplies the j_2-th component by $e^{-ix\lambda_{j_2}}$. Since $e^{-ix\lambda_{j_2}} e^{-ix\lambda_{j_1}} = e^{-ix(\lambda_{j_1}+\lambda_{j_2})}$, we obtain

$$\hat{\phi}_{j_2}^{(2)} = e^{-ix\lambda_{j_2}} \phi_{j_2}^{(2)} = \sum_{j_1=1}^{d} W_{j_2 j_1}^{(2)} W_{j_1 1}^{(1)}\, e^{-ix(\lambda_{j_1}+\lambda_{j_2})}. \tag{4.26}$$

Finally, applying the third block yields

$$\psi_m(x) = \sum_{j_2=1}^{d} W_{m j_2}^{(3)} \hat{\phi}_{j_2}^{(2)} = \sum_{j_1=1}^{d} \sum_{j_2=1}^{d} W_{m j_2}^{(3)} W_{j_2 j_1}^{(2)} W_{j_1 1}^{(1)}\, e^{-ix(\lambda_{j_1}+\lambda_{j_2})}. \tag{4.27}$$

Each term factorizes into two parts:

1. an input-independent coefficient, $W_{m j_2}^{(3)} W_{j_2 j_1}^{(2)} W_{j_1 1}^{(1)}$, determined by the data-independent blocks; and

2. an input-dependent phase, $e^{-ix(\lambda_{j_1}+\lambda_{j_2})}$, whose frequency is a sum of eigenvalues of the encoding Hamiltonian, one from each encoding layer.

Step 4: General number of encoding layers. The same pattern persists for any number of layers: each additional encoding gate contributes one phase factor, and each additional block introduces one summation index. Formally, the result follows by induction on L. If the components after block $W^{(l)}$ are sums over $j_1, \ldots, j_{l-1}$ of products of block entries multiplied by $e^{-ix(\lambda_{j_1}+\cdots+\lambda_{j_{l-1}})}$, then the subsequent encoding gate multiplies component j_l by $e^{-ix\lambda_{j_l}}$, and block $W^{(l+1)}$ introduces a sum over j_l; this reproduces the same form with l replaced by $l+1$. For L encoding layers we therefore have

$$\psi_m(x) = \sum_{j_1,\ldots,j_L=1}^{d} A_{m,j_1,\ldots,j_L}\, e^{-ix(\lambda_{j_1}+\cdots+\lambda_{j_L})}, \tag{4.28}$$

where the coefficients are products of entries of the data-independent blocks:

$$A_{m,j_1,\ldots,j_L} = W^{(L+1)}_{mj_L} W^{(L)}_{j_L j_{L-1}} \cdots W^{(2)}_{j_2 j_1} W^{(1)}_{j_1 1}. \tag{4.29}$$

To shorten the notation, we collect the indices into a multi-index and define the corresponding eigenvalue sum:

$$\mathbf{j} = (j_1, \ldots, j_L) \in [d]^L, \qquad \Lambda_{\mathbf{j}} = \lambda_{j_1} + \lambda_{j_2} + \cdots + \lambda_{j_L}, \tag{4.30}$$

where $[d] = \{1, \ldots, d\}$, so that $[d]^L$ is the set of all d^L index tuples of length L. The amplitudes then take the compact form

$$\psi_m(x) = \sum_{\mathbf{j} \in [d]^L} A_{m,\mathbf{j}}\, e^{-ix\Lambda_{\mathbf{j}}}. \tag{4.31}$$

In words, regardless of the depth of the circuit, every amplitude is a sum of terms of the form (circuit coefficient) $\times\, e^{-ix\cdot(\text{eigenvalue sum})}$. The coefficients depend on the trainable blocks, whereas the eigenvalue sums depend only on the encoding Hamiltonian and on L.

For example, with $d = 2$ and $L = 2$, the multi-index ranges over

$$\mathbf{j} \in \{(1,1), (1,2), (2,1), (2,2)\}, \tag{4.32}$$

and Eq. (4.31) contains four terms:

$$\psi_m(x) = A_{m,(1,1)} e^{-2ix\lambda_1} + A_{m,(1,2)} e^{-ix(\lambda_1+\lambda_2)} + A_{m,(2,1)} e^{-ix(\lambda_2+\lambda_1)} + A_{m,(2,2)} e^{-2ix\lambda_2}. \tag{4.33}$$

The multi-indices $(1,2)$ and $(2,1)$ yield the same eigenvalue sum, so the corresponding terms share a frequency. Such coincidences are common and will be used in Step 6 to combine terms.

Step 5: Measurement. The amplitudes contain frequencies $\Lambda_{\mathbf{j}}$, whereas the frequencies of the model output turn out to be differences $\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}}$. This change originates in the measurement: the expectation value in Eq. (4.11) is quadratic in the state and involves both the amplitudes and their complex conjugates. Writing

$$|\psi(x)\rangle = \begin{bmatrix} \psi_1(x) \\ \vdots \\ \psi_d(x) \end{bmatrix}, \qquad \langle\psi(x)| = \begin{bmatrix} \psi_1^*(x) & \cdots & \psi_d^*(x) \end{bmatrix}, \tag{4.34}$$

the expectation value becomes

$$f(x) = \sum_{m=1}^{d} \sum_{n=1}^{d} \psi_m^*(x)\, O_{mn}\, \psi_n(x). \tag{4.35}$$

For a single qubit ($d = 2$), for instance,

$$\begin{aligned} f(x) &= \begin{bmatrix} \psi_1^* & \psi_2^* \end{bmatrix} \begin{bmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{bmatrix} \begin{bmatrix} \psi_1 \\ \psi_2 \end{bmatrix} \\ &= \psi_1^* O_{11} \psi_1 + \psi_1^* O_{12} \psi_2 + \psi_2^* O_{21} \psi_1 + \psi_2^* O_{22} \psi_2. \end{aligned} \tag{4.36}$$

The off-diagonal entries of O couple different amplitudes through cross terms. Even when O is diagonal (e.g., $O = Z$), the output contains the products $\psi_m^* \psi_m$ and therefore remains quadratic in the amplitudes.

To evaluate Eq. (4.35), we require the complex conjugate of Eq. (4.31). Because the eigenvalue sums $\Lambda_{\mathbf{k}}$ are real,

$$\psi_m^*(x) = \sum_{\mathbf{k} \in [d]^L} A_{m,\mathbf{k}}^* \, e^{ix\Lambda_{\mathbf{k}}}. \tag{4.37}$$

Conjugation has two effects: each coefficient is replaced by its complex conjugate, and the sign of the exponent changes from negative to positive. We also rename the summation multi-index to $\mathbf{k}$; it ranges over the same set $[d]^L$ as $\mathbf{j}$, but the two sums are independent. Substituting both expansions into Eq. (4.35) gives

$$\begin{aligned} f(x) &= \sum_{m,n} \left(\sum_{\mathbf{k}} A_{m,\mathbf{k}}^* \, e^{ix\Lambda_{\mathbf{k}}} \right) O_{mn} \left(\sum_{\mathbf{j}} A_{n,\mathbf{j}} \, e^{-ix\Lambda_{\mathbf{j}}} \right) \\ &= \sum_{m,n} \sum_{\mathbf{k},\mathbf{j}} A_{m,\mathbf{k}}^* \, O_{mn} \, A_{n,\mathbf{j}} \, e^{ix\Lambda_{\mathbf{k}}} e^{-ix\Lambda_{\mathbf{j}}}. \end{aligned} \tag{4.38}$$

Using $e^{ix\Lambda_{\mathbf{k}}} e^{-ix\Lambda_{\mathbf{j}}} = e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}$ and exchanging the order of summation, we obtain

$$f(x) = \sum_{\mathbf{k},\mathbf{j} \in [d]^L} \left(\sum_{m,n} A_{m,\mathbf{k}}^* \, O_{mn} \, A_{n,\mathbf{j}} \right) e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}. \tag{4.39}$$

Defining the input-independent coefficients

$$a_{\mathbf{k},\mathbf{j}} = \sum_{m,n} A_{m,\mathbf{k}}^* \, O_{mn} \, A_{n,\mathbf{j}}, \tag{4.40}$$

the model output becomes

$$f(x) = \sum_{\mathbf{k},\mathbf{j} \in [d]^L} a_{\mathbf{k},\mathbf{j}} \, e^{ix(\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}})}. \tag{4.41}$$

Thus the output is a sum of terms of the form (coefficient) $\times \, e^{ix \cdot (\text{difference of eigenvalue sums})}$.

Step 6: Frequency spectrum and Fourier coefficients. Comparing each term in Eq. (4.41) with the Fourier basis function $e^{i\omega x}$, we identify its frequency as

$$\omega = \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \sum_{l=1}^{L} \lambda_{k_l} - \sum_{l=1}^{L} \lambda_{j_l}. \tag{4.42}$$

The set of frequencies accessible to the model, its *frequency spectrum*, is therefore

$$\Omega = \left\{ \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} \ : \ \mathbf{k}, \mathbf{j} \in [d]^L \right\}. \tag{4.43}$$

To summarize the notation: λ denotes an eigenvalue of the encoding Hamiltonian, Λ a sum of eigenvalues across the L encoding layers, and ω a difference of two such sums.

As anticipated in Step 4, distinct pairs of multi-indices may produce the same frequency. If, for example, $\Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \Lambda_{\mathbf{k}'} - \Lambda_{\mathbf{j}'} = \omega$ for two different pairs $(\mathbf{k}, \mathbf{j})$ and $(\mathbf{k}', \mathbf{j}')$, the corresponding terms combine as

$$a_{\mathbf{k},\mathbf{j}}\, e^{i\omega x} + a_{\mathbf{k}',\mathbf{j}'}\, e^{i\omega x} = (a_{\mathbf{k},\mathbf{j}} + a_{\mathbf{k}',\mathbf{j}'})\, e^{i\omega x}. \tag{4.44}$$

Collecting all pairs that share a frequency, we define the Fourier coefficient associated with $\omega \in \Omega$ as

$$c_\omega = \sum_{\substack{\mathbf{k},\mathbf{j} \in [d]^L \\ \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} = \omega}} a_{\mathbf{k},\mathbf{j}}, \tag{4.45}$$

and arrive at the Fourier representation of the QNN:

$$\boxed{f(x) = \sum_{\omega \in \Omega} c_\omega\, e^{i\omega x}} \tag{4.46}$$

Since Ω contains at most d^{2L} elements, this is a finite sum. When the eigenvalue differences are commensurate, as for the Pauli encodings discussed below, all frequencies are integer multiples of a fundamental frequency and Eq. (4.46) is a truncated Fourier series in the usual sense; in general, it is a finite sum of complex exponentials.

The representation has two further properties. First, the spectrum is symmetric and contains zero: if $\omega = \Lambda_{\mathbf{k}} - \Lambda_{\mathbf{j}} \in \Omega$, then $-\omega = \Lambda_{\mathbf{j}} - \Lambda_{\mathbf{k}} \in \Omega$, and $\mathbf{k} = \mathbf{j}$ gives $\omega = 0$. Second, because O is Hermitian ($O^*_{mn} = O_{nm}$), Eq. (4.40) implies

$$a^*_{\mathbf{k},\mathbf{j}} = \sum_{m,n} A_{m,\mathbf{k}}\, O^*_{mn}\, A^*_{n,\mathbf{j}} = \sum_{m,n} A^*_{n,\mathbf{j}}\, O_{nm}\, A_{m,\mathbf{k}} = a_{\mathbf{j},\mathbf{k}}, \tag{4.47}$$

and summing over all pairs with frequency ω gives $c_{-\omega} = c^*_\omega$. Consequently, $f(x)$ is real-valued, as an expectation value of a Hermitian observable must be, and can equivalently be written as

$$f(x) = c_0 + \sum_{\omega \in \Omega,\, \omega > 0} 2\,|c_\omega| \cos(\omega x + \arg c_\omega)\,. \tag{4.48}$$

Roles of the encoding and trainable blocks. The blocks $W^{(l)}$ do not appear in the frequency spectrum (4.43), which is determined entirely by the eigenvalues of the encoding Hamiltonian and the number of encoding layers L. This is because the blocks are independent of x: they enter only through the coefficients $A_{m,\mathbf{j}}$ in Eq. (4.29), and hence only through $a_{\mathbf{k},\mathbf{j}}$ and c_ω, while leaving the exponential factors $e^{i\omega x}$ unchanged. Training therefore adjusts the amplitudes and phases of a fixed set of frequencies but cannot introduce frequencies outside Ω. The extent to which the coefficients themselves can be controlled depends on the expressivity of the trainable blocks. If different encoding Hamiltonians $H^{(l)}$ are used in different layers, the same derivation applies with $\Lambda_{\mathbf{j}} = \sum_l \lambda^{(l)}_{j_l}$, where $\lambda^{(l)}_j$ are the eigenvalues of $H^{(l)}$.

Example: Pauli-rotation encoding. For single-qubit encoding with $R_x(x)$, we have $H = \frac{1}{2}X$ with eigenvalues $\pm\frac{1}{2}$. Each eigenvalue sum $\Lambda_{\mathbf{j}}$ is a sum of L terms equal to $\pm\frac{1}{2}$ and

therefore takes values in $\{-\frac{L}{2}, -\frac{L}{2}+1, \ldots, \frac{L}{2}\}$. Their differences are the integers from $-L$ to L, so that

$$\Omega = \{-L, \ldots, -1, 0, 1, \ldots, L\}, \qquad f(x) = \sum_{\omega=-L}^{L} c_\omega \, e^{i\omega x}. \tag{4.49}$$

A model that encodes x L times through a single-qubit Pauli rotation is thus a truncated Fourier series of degree L. In particular, with a single encoding layer ($L = 1$), the model reduces to $f(x) = c_0 + 2|c_1| \cos(x + \arg c_1)$, a single sinusoid, irrespective of how expressive the trainable blocks are. Accessing higher frequencies requires encoding the input repeatedly, either sequentially (data re-uploading, larger L) or in parallel; for instance, applying $R_x(x)$ simultaneously to N qubits gives $H = \frac{1}{2}\sum_{q=1}^{N} X_q$, whose eigenvalues range over $\{-\frac{N}{2}, \ldots, \frac{N}{2}\}$, so a single such layer already yields the spectrum $\{-N, \ldots, N\}$.

4.3 Expressivity and Frequency Expansion

Section 4.2 established that a QNN computes the finite Fourier series of Eq. (4.46), in which the frequency spectrum Ω is fixed by the eigenvalues of the encoding Hamiltonians and the number of encoding gates, while the trainable blocks determine only the coefficients c_ω. A direct consequence is that the model can represent only functions whose frequency content lies within Ω: no choice of trainable parameters can produce a frequency outside this set. A general target function, however, may contain many frequency components, some of them high. For a QNN to act as a universal function approximator, two conditions must therefore hold: (i) the spectrum Ω must be rich enough to contain the frequencies required by the target, and (ii) the trainable blocks must be expressive enough to realize the required coefficients c_ω. Schuld et al. [35] showed that, when both conditions are met and the spectrum is allowed to grow, a QNN can approximate any square-integrable function on a bounded interval arbitrarily well. This subsection addresses condition (i).

Because Ω consists of differences of eigenvalue sums, it can be enlarged only by enlarging the set of eigenvalue sums. We examine three strategies for doing so:

1. *parallel encoding*, in which the input is encoded on more qubits (increasing the circuit width);
2. *serial encoding*, or data re-uploading, in which the input is encoded repeatedly on the same qubits (increasing the circuit depth); and
3. *input scaling*, in which the input is multiplied by constant coefficients before encoding, which relocates the frequencies rather than adding encoding gates.

Throughout this subsection, the encoding gates are single-qubit Pauli rotations. Every Pauli rotation $R_P(x) = e^{-i\frac{x}{2}P}$, with $P \in \{X, Y, Z\}$, has encoding Hamiltonian $H = \frac{1}{2}P$ and eigenvalues $\pm\frac{1}{2}$. For concreteness, we write all examples with $P = X$.

4.3.1 Parallel Encoding: Increasing the Number of Encoding Qubits

One qubit. Consider first a single encoding qubit with

$$S(x) = R_x(x) = e^{-ixH}, \qquad H = \frac{1}{2}X = \frac{1}{2}\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{4.50}$$

The eigenvalues of H are $\mathrm{spec}(H) = \{-\frac{1}{2}, \frac{1}{2}\}$, and their pairwise differences give the spectrum $\Omega = \{-1, 0, 1\}$. By the conjugate symmetry $c_{-\omega} = c_\omega^*$ established in Section 4.2, the model therefore reduces to a single sinusoid, $f(x) = c_0 + 2|c_1| \cos(x + \arg c_1)$.

Two qubits. Suppose now that x is encoded once on each of two qubits. Let X_q denote the Pauli-X operator acting on encoding qubit q (and the identity on the other qubit). Following the little-endian convention of Section 3, qubit 1 is the rightmost tensor factor, so that $X_1 = I \otimes X$ and $X_2 = X \otimes I$. The encoding gate is

$$S(x) = R_x(x) \otimes R_x(x) = e^{-i\frac{x}{2}X_1}\, e^{-i\frac{x}{2}X_2} = e^{-ixH}, \qquad H = \frac{1}{2}\left(X_1 + X_2\right). \tag{4.51}$$

The first equality uses $e^A \otimes e^B = (e^A \otimes I)(I \otimes e^B)$ together with $e^A \otimes I = e^{A \otimes I}$; the second holds because X_1 and X_2 commute, so that $e^A e^B = e^{A+B}$. Since X_1 and X_2 commute, they are simultaneously diagonalized by the product basis $\{|+\rangle, |-\rangle\}^{\otimes 2}$, on which X_q takes the value $\epsilon_q \in \{-1, +1\}$. The eigenvalues of H are therefore $\frac{1}{2}(\epsilon_1 + \epsilon_2)$, namely -1, 0 (twice), and 1. The set of distinct eigenvalues is $\{-1, 0, 1\}$, and its pairwise differences give the enlarged spectrum

$$\Omega = \{-2, -1, 0, 1, 2\}. \tag{4.52}$$

N qubits. The same argument applies when x is encoded once on each of N qubits. The encoding Hamiltonian is

$$H = \frac{1}{2}\sum_{q=1}^{N} X_q, \tag{4.53}$$

with eigenvalues $\frac{1}{2}\sum_q \epsilon_q$, $\epsilon_q \in \{-1, +1\}$. If k of the N signs are negative, the eigenvalue equals $\frac{1}{2}\Big((N-k) - k\Big)$, so the distinct eigenvalues are

$$\lambda_k = \frac{N}{2} - k, \qquad k = 0, 1, \ldots, N, \tag{4.54}$$

where λ_k has multiplicity $\binom{N}{k}$. Consecutive eigenvalues differ by one, and the largest and smallest differ by N; hence

$$\Omega = \{-N, -N+1, \ldots, -1, 0, 1, \ldots, N-1, N\}. \tag{4.55}$$

Encoding on N qubits thus gives access to $2N + 1$ integer frequencies. Because the spectrum is symmetric and $c_{-\omega} = c_\omega^*$, only the $N + 1$ non-negative frequencies $\{0, 1, \ldots, N\}$ are independent; the model is a real trigonometric polynomial of degree N. The number of accessible frequencies grows linearly with the number of encoding qubits.

Example. Figure 12 shows a five-qubit QNN. The unscaled input x is encoded once on each qubit through a Pauli rotation, and the circuit begins directly with this encoding layer ($W^{(1)} = I$, which does not affect the spectrum). The encoding layer is followed by two trainable layers. Each trainable layer applies parameterized single-qubit rotations R_x, R_y, and R_z to every qubit, followed by a ring of CNOT gates that couples each qubit to its

neighbor and the last qubit back to the first, thereby generating entanglement across the register. By Eq. (4.55) with $N = 5$, the accessible frequencies are $\{0, \pm1, \pm2, \pm3, \pm4, \pm5\}$. Which of these frequencies carry non-zero coefficients in practice depends on the trainable layers.

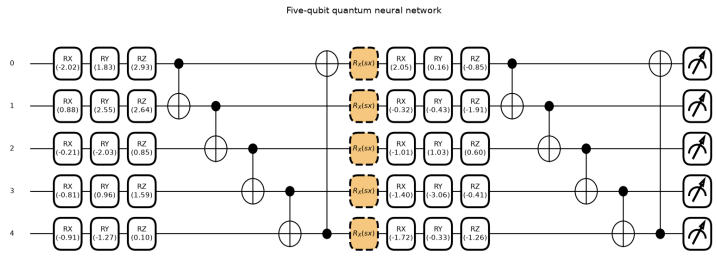


Figure 12: Architecture of the five-qubit QNN, consisting of a single encoding layer followed by two layers of parameterized rotation gates, each followed by a CNOT ring.

4.3.2 Serial Encoding: Data Re-uploading

Parallel encoding requires one qubit per encoding gate. When the number of available qubits is limited, a model with only a single qubit can still access multiple frequencies by *data re-uploading*: the same input x is encoded repeatedly on the same qubit, with trainable blocks interleaved between the encodings. For a single qubit with L encoding layers, the circuit takes the form of Eq. (4.7),

$$U(x) = W^{(L+1)} S^{(L)}(x)\, W^{(L)} \cdots W^{(2)} S^{(1)}(x)\, W^{(1)}, \tag{4.56}$$

with model output $f(x) = \langle 0|\, U^\dagger(x)\, O\, U(x)\, |0\rangle$. Here $S^{(l)}(x) = e^{-ixH_l}$ denotes the l-th encoding gate; in this subsection all encoding gates are identical, $S^{(l)}(x) = R_x(x)$ with $H_l = \frac{1}{2}X$. We allow the encoding Hamiltonians to depend on l in anticipation of Section 4.3.3.

As noted in Section 4.2, the derivation of the Fourier representation carries over to layer-dependent encoding Hamiltonians. Each eigenvalue sum then takes one eigenvalue from each layer, and we denote the set of all such sums by

$$\mathcal{E} = \left\{ \sum_{l=1}^{L} \lambda^{(l)} \;\middle|\; \lambda^{(l)} \in \mathrm{spec}(H_l) \right\}. \tag{4.57}$$

The spectrum consists of all pairwise differences of elements of $\mathcal{E}$:

$$\Omega = \{\Lambda - \Lambda' \mid \Lambda, \Lambda' \in \mathcal{E}\}. \tag{4.58}$$

Two encoding layers. For $L = 2$ with $H_1 = H_2 = \frac{1}{2}X$, each layer contributes an eigenvalue $\pm\frac{1}{2}$, so

$$\mathcal{E} = \left\{ \pm\tfrac{1}{2} \pm \tfrac{1}{2} \right\} = \{-1, 0, 1\}, \qquad \Omega = \{-2, -1, 0, 1, 2\}. \tag{4.59}$$

This is exactly the spectrum obtained with two qubits in Section 4.3.1.

L encoding layers. For general L with $H_l = \frac{1}{2}X$ for all l, each eigenvalue sum consists of L terms equal to $\pm\frac{1}{2}$. If k of these terms are negative, the sum equals $\frac{L}{2} - k$, so that

$$\mathcal{E} = \left\{-\frac{L}{2}, -\frac{L}{2}+1, \ldots, \frac{L}{2}\right\}, \tag{4.60}$$

and by Eq. (4.58),

$$\Omega = \{0, \pm 1, \pm 2, \ldots, \pm L\}. \tag{4.61}$$

Comparison with parallel encoding. A single qubit re-uploaded L times and $N = L$ qubits each encoded once have identical spectra. The reason is that, in both cases, every eigenvalue sum is a sum of $\pm\frac{1}{2}$ terms, one contributed by each encoding gate; the spectrum depends only on the total number of encoding gates and not on whether they are arranged in parallel or in series. The two strategies differ in their resource cost: parallel encoding consumes qubits, whereas serial encoding consumes circuit depth, which increases execution time and exposure to noise on hardware. The strategies can also be combined: encoding on N qubits in each of L layers yields NL encoding gates and hence the spectrum $\{-NL, \ldots, NL\}$.

4.3.3 Accessing High Frequencies through Input Scaling

In the encodings considered so far, the spectrum consists of consecutive integers up to the total number of encoding gates. Many applications, however, require high-frequency components to be captured. A target function containing the frequency $\omega = 1000$ would then require 1000 encoding qubits, or 1000 re-uploading layers. This cost can be avoided by multiplying the input x by a constant coefficient before it is encoded. Scaling the input rescales the encoding Hamiltonian, since $R_x(ax) = e^{-ix\frac{a}{2}X}$ corresponds to $H = \frac{a}{2}X$ with eigenvalues $\pm\frac{a}{2}$. The scaling coefficients may be fixed in advance or treated as trainable parameters; in the latter case, the spectrum itself becomes trainable.

Two qubits. Consider a two-qubit QNN in which the input to the first qubit is scaled by $a \in \mathbb{R}$ and the input to the second qubit by $b \in \mathbb{R}$. The encoding gate is

$$S(x) = R_x(bx) \otimes R_x(ax) = e^{-ixH}, \qquad H = \frac{1}{2}\left(aX_1 + bX_2\right), \tag{4.62}$$

where the second equality follows exactly as in Section 4.3.1. In the product basis $\{|+\rangle, |-\rangle\}^{\otimes 2}$, the eigenvalues of H are $\frac{1}{2}(\epsilon_1 a + \epsilon_2 b)$ with $\epsilon_1, \epsilon_2 \in \{-1, +1\}$:

$$\operatorname{spec}(H) = \left\{\frac{a+b}{2}, \frac{a-b}{2}, \frac{-a+b}{2}, \frac{-a-b}{2}\right\}. \tag{4.63}$$

The difference between two eigenvalues, labeled by signs (ϵ_1, ϵ_2) and $(\epsilon_1', \epsilon_2')$, is

$$\frac{\epsilon_1 - \epsilon_1'}{2}\, a + \frac{\epsilon_2 - \epsilon_2'}{2}\, b, \tag{4.64}$$

where each factor $\frac{1}{2}(\epsilon_q - \epsilon_q')$ takes a value in $\{-1, 0, 1\}$. The accessible frequencies are therefore

$$\Omega = \{0,\ \pm a,\ \pm b,\ \pm(a+b),\ \pm(a-b)\}. \tag{4.65}$$

High frequencies thus become accessible without additional qubits. For instance, $\omega = 1000$ lies in Ω if $a = 1000$, or if a and b are chosen such that $a + b = 1000$ or $a - b = 1000$. Setting $a = b = 1$ recovers the unscaled spectrum $\{-2, \ldots, 2\}$.

N qubits. The argument extends directly to N qubits, where qubit q encodes $a_q x$. The encoding Hamiltonian, its eigenvalues, and the resulting spectrum are

$$\begin{aligned} H &= \frac{1}{2}\sum_{q=1}^{N} a_q X_q, \\ \lambda &= \frac{1}{2}\sum_{q=1}^{N} \epsilon_q a_q, \qquad \epsilon_q \in \{-1, +1\}, \\ \Omega &= \left\{ \sum_{q=1}^{N} \nu_q a_q \;\middle|\; \nu_q \in \{-1, 0, 1\} \right\}, \end{aligned} \tag{4.66}$$

where $\nu_q = \frac{1}{2}(\epsilon_q - \epsilon'_q)$ as in the two-qubit case. The unscaled encoding corresponds to $a_q = 1$ for all q, which recovers Eq. (4.55). For example, consider the scaled encoding with $N = 5$ and $a_q = q$; since every integer from 0 to 15 is a sum of distinct elements of $\{1, \ldots, 5\}$, this choice yields $\Omega = \{-15, \ldots, 15\}$.

The frequency budget. Input scaling changes *where* the frequencies lie but not *how many* there can be. Since each ν_q in Eq. (4.66) takes one of three values, $|\Omega| \leq 3^N$ for any choice of coefficients. The choice of $\{a_q\}$ therefore trades density against reach. With $a_q = 1$, the spectrum is the dense range $\{-N, \ldots, N\}$, and many sign combinations produce the same frequency. With $a_q = q$, it extends to $\pm N(N+1)/2$. With $a_q = 3^{q-1}$, the bound is attained: every integer in $[-\frac{3^N-1}{2}, \frac{3^N-1}{2}]$ has a unique balanced-ternary representation, so Ω contains all of them, for example $\{-121, \ldots, 121\}$ when $N = 5$. Conversely, coefficients chosen to reach a single high frequency leave gaps: frequencies between those in Ω cannot be represented. If the coefficients are not commensurate, the frequencies are no longer integer multiples of a common fundamental, and f is not periodic.

Single qubit with scaled re-uploading. High frequencies can also be accessed with a single qubit by assigning each re-uploading layer in Eq. (4.56) its own coefficient. The l-th encoding gate is then

$$S^{(l)}(x) = R_x(s_l x) = e^{-ixH_l}, \qquad H_l = \frac{s_l}{2}X, \qquad \mathrm{spec}(H_l) = \left\{-\frac{s_l}{2}, \frac{s_l}{2}\right\}, \tag{4.67}$$

where $s_l \in \mathbb{R}$ is the scaling coefficient of the l-th layer. The set of eigenvalue sums is

$$\mathcal{E} = \left\{ \frac{1}{2}\sum_{l=1}^{L} \epsilon_l s_l \;\middle|\; \epsilon_l \in \{-1, +1\} \right\}, \tag{4.68}$$

and, by Eq. (4.58) and the same argument as above,

$$\Omega = \left\{ \sum_{l=1}^{L} \nu_l s_l \;\middle|\; \nu_l \in \{-1, 0, 1\} \right\}. \tag{4.69}$$

This expression has the same form as Eq. (4.66) with $N = L$ and $a_q = s_q$. As in the unscaled case, parallel encoding across qubits and serial encoding through re-uploading give access to the same set of frequencies, and the frequency budget $|\Omega| \leq 3^L$ applies.

Example. With two re-uploading layers and coefficients $s_1 = 1$ and $s_2 = 1000$, the circuit

$$U(x) = W^{(3)}\, R_x(1000x)\, W^{(2)}\, R_x(x)\, W^{(1)} \tag{4.70}$$

has the spectrum

$$\Omega = \{0, \pm 1, \pm 999, \pm 1000, \pm 1001\}. \tag{4.71}$$

A single qubit can therefore represent both the frequency $\omega = 1$ and the frequency $\omega = 1000$. The combinations $\omega = 999$ and $\omega = 1001$ are accessible as well, whereas the intermediate frequencies $2, \ldots, 998$ are not, in accordance with the frequency budget discussed above.

Summary. Table 1 summarizes the three strategies. In every case, the spectrum is determined by the set of eigenvalue sums of the encoding Hamiltonians, and the trainable blocks act only on the coefficients.

Table 1: Frequency spectra of the encoding strategies discussed in this section, for Pauli-rotation encodings. In the scaled cases, $\nu_q, \nu_l \in \{-1, 0, 1\}$.

Strategy	Encoding	Spectrum Ω	Resource cost
Parallel	N qubits, once each	$\{-N, \ldots, N\}$	N qubits
Serial (re-uploading)	1 qubit, L layers	$\{-L, \ldots, L\}$	depth L
Scaled parallel	N qubits, inputs $a_q x$	$\{\sum_q \nu_q a_q\}$	N qubits
Scaled serial	1 qubit, inputs $s_l x$	$\{\sum_l \nu_l s_l\}$	depth L

4.4 Training and Barren Plateaus

The preceding subsections characterized which functions a QNN can represent. Whether training can find a good function within this class is a separate question, which we discuss briefly here.

The hybrid training loop. A QNN is trained in the same way as a classical network. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, a loss $\mathcal{L}(\boldsymbol{\theta})$, such as the mean squared error of Eq. (2.5), is minimized by gradient descent, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$, with learning rate η. The quantum device evaluates the model outputs and their derivatives, and a classical computer updates the parameters [21]. By the chain rule, the loss gradient requires the derivatives $\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)/\partial \theta_k$ of the circuit output. On hardware, these cannot be obtained by backpropagation, which requires access to intermediate states that cannot be read out without measurement (Section 3.7).

The parameter-shift rule. The Fourier analysis of Section 4.2 provides these derivatives directly. Consider a parameter θ_k that enters a single gate $R_P(\theta_k) = e^{-i\theta_k P/2}$, with $P \in \{X, Y, Z\}$, and hold the input and all other parameters fixed. The circuit then has the form of Eq. (4.7) with θ_k in place of x, a single encoding layer ($L = 1$), and Hamiltonian $H = \frac{1}{2}P$ with eigenvalues $\pm\frac{1}{2}$. By the result of Section 4.2, the output is therefore a single sinusoid in θ_k,

$$f(\theta_k) = a + b\cos\theta_k + c\sin\theta_k, \tag{4.72}$$

where a, b, and c depend on the input and on the remaining parameters. Evaluating this function at $\theta_k \pm \frac{\pi}{2}$ and subtracting gives

$$\frac{\partial f}{\partial \theta_k} = \frac{1}{2}\left[f\left(\theta_k + \frac{\pi}{2}\right) - f\left(\theta_k - \frac{\pi}{2}\right)\right]. \tag{4.73}$$

Unlike a finite-difference approximation, this expression is exact: the derivative is obtained from two evaluations of the same circuit with one angle shifted by $\pm\frac{\pi}{2}$ [25, 45]. When a parameter enters several gates, the rule is applied to each gate and the results are summed. Applied to the encoding gates, the same rule yields exact derivatives with respect to the input x, which Section 5.4 uses to solve differential equations. The cost, however, differs from that of backpropagation. A model with p parameters requires $2p$ circuit evaluations per data point for a single gradient, each estimated from many shots (Section 3.7), whereas backpropagation obtains all p derivatives at a cost comparable to a single forward pass.

Barren plateaus. Exact gradients are useful only if they are large enough to be estimated. McClean et al. showed that, for sufficiently random parameterized circuits, the gradient has zero mean and a variance that vanishes exponentially with the number of qubits [31]. The loss landscape is then almost flat almost everywhere, a phenomenon known as a *barren plateau*. Because each gradient component is estimated with a standard error of order $1/\sqrt{N}$ from N shots (Section 3.7), resolving an exponentially small gradient requires exponentially many shots, and training stalls. The effect also depends on the cost function: global observables, which act on all qubits, can produce barren plateaus even in shallow circuits, whereas local observables avoid them for sufficiently shallow circuits [46]. Hardware noise can induce barren plateaus of its own, independently of how the parameters are initialized [47].

Common mitigation strategies restrict the circuit, for example through shallow and structured architectures, local observables, or initializations that make the circuit close to the identity at the start of training [48]. Such restrictions, however, also limit which Fourier coefficients the circuit can reach (Section 5.7), and the structure that allows a circuit to avoid barren plateaus may also allow it to be simulated classically [34]. Trainability and expressivity therefore cannot be designed independently: a QNN must be expressive enough to represent the target, yet structured enough to be trained.

5 Applications of Quantum Neural Networks

The preceding sections established a precise characterization of what a QNN computes: a finite Fourier series whose frequencies are fixed by the data encoding and whose coefficients

are learned by the trainable blocks. This section uses that characterization to address a practical question: *which tasks and applications are QNNs well suited to?* Rather than cataloging every domain in which QNNs have been tried, we derive suitability criteria from the Fourier picture and examine representative application areas against them, distinguishing throughout what QNNs can represent in principle from what has been demonstrated in practice.

5.1 From Fourier Properties to Task Suitability

Multivariate inputs. Section 4.2 considered a scalar input. Most applications, however, involve an input vector $\mathbf{x} = (x_1, \ldots, x_D) \in \mathbb{R}^D$. When each feature x_k is encoded through its own Hamiltonian-generated gates, the same derivation yields a multivariate Fourier series [35, 49],

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{\boldsymbol{\omega} \in \Omega} c_{\boldsymbol{\omega}}(\boldsymbol{\theta})\, e^{i\boldsymbol{\omega}\cdot\mathbf{x}}, \qquad \Omega = \Omega_1 \times \Omega_2 \times \cdots \times \Omega_D, \tag{5.1}$$

where Ω_k is the spectrum generated by the encodings of feature x_k, and $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_D)$ is a frequency vector. If every feature is encoded so that $\Omega_k = \{-K, \ldots, K\}$, the model contains $(2K+1)^D$ frequency vectors. Both the spectrum and the number of coefficients to be controlled therefore grow exponentially with the input dimension, and Casas and Cervera-Lierta showed that, for several common circuit families, the degrees of freedom required to fit a general multidimensional Fourier series grow faster than those available in the circuit [49].

Suitability criteria. Combining Eq. (5.1) with the results of Sections 4.2 and 4.3, we identify five properties that make a task a natural fit for a QNN:

1. **Low input dimension.** Small D keeps the spectrum and the encoding cost manageable.
2. **Known, sparse, or structured spectral content.** When the dominant frequencies of the target are known in advance, the encoding can place Ω exactly where the signal lives, using the scaling strategies of Section 4.3.3.
3. **Smoothness or periodicity.** A truncated Fourier series represents smooth and periodic functions efficiently, whereas discontinuities produce persistent oscillations near the jump (the Gibbs phenomenon [44]).
4. **Modest data volume.** Every data point requires many circuit executions to estimate an expectation value.
5. **Quantum-native data.** When the input is itself a quantum state, the encoding problem disappears. This criterion lies outside the Fourier picture, but it is where the case for quantum advantage is strongest.

The following subsections examine application domains in roughly decreasing order of fit.

5.2 Implicit Neural Representations: Images, Audio, and Beyond

An implicit neural representation (INR) replaces a discrete signal, such as a grid of pixels or a sequence of audio samples, with a continuous function learned by a neural network. The network maps coordinates to signal values: $(x, y) \mapsto$ RGB for an image, $t \mapsto$ amplitude for an audio clip, $(x, y, z) \mapsto$ signed distance for a 3D shape [50], or position and viewing direction to color and density in neural radiance fields [51].

INRs satisfy the first two criteria almost ideally, and the third whenever the signal is smooth: the input dimension is small ($D = 1$ to 5), and the quality of an INR is judged precisely by how well it reproduces the signal's spectral content. This is also where classical networks struggle most visibly, since spectral bias causes ReLU multilayer perceptrons to produce blurry reconstructions that miss fine detail [36]. The leading classical remedies are themselves Fourier constructions: Fourier feature mappings pass the coordinates through sinusoids before an MLP [37], and SIREN uses sinusoidal activations throughout [38]. A QNN provides this structure natively, with a spectrum designed through the encoding.

This connection has been explored directly. Zhao et al. proposed the Quantum Implicit Representation Network (QIREN), which combines data re-uploading circuits with classical layers, and reported that it outperforms classical INR baselines on image and sound representation, particularly in modeling high-frequency components, while using fewer parameters; they also explored image super-resolution and image generation [52]. Because these results were obtained with classical simulation of small circuits, they establish representational promise rather than a hardware advantage.

Two obstacles remain. First, INRs are trained on every coordinate of the signal: a 256×256 image already contains 65,536 training points, each requiring many circuit executions on hardware. Second, sharp edges give rise to Gibbs oscillations in any truncated Fourier representation. Moreover, with integer spectra the model is 2π-periodic, so coordinates should be mapped to a sub-interval of $[0, 2\pi)$, as recommended for angle encoding in Section 3.3, to avoid forcing the two ends of the signal to agree. Nevertheless, among classical-data applications, INRs are arguably the most natural match between task requirements and the QNN's inductive bias.

5.3 Audio and Time-Series Signals

Audio and many time series are superpositions of oscillations. Voiced and musical sounds consist of a fundamental frequency and its harmonics, and many time series exhibit known periodicities, such as daily and weekly cycles in electricity load, traffic, and transaction volumes. Classical forecasting practice already models such seasonality with explicit Fourier terms [53]. For a QNN, known periodicities translate directly into encoding design: choosing the scaling coefficients of Section 4.3.3 so that Ω contains the seasonal frequencies and their low harmonics.

Several studies have applied QNNs to such signals. Emmanoulopoulos and Dimoska reported that parameterized quantum circuits with only a few parameters forecast periodic signals with small noise components about as well as classical bidirectional LSTM networks [54]. In speech recognition, Yang et al. used quantum circuits as convolutional feature extractors on Mel-spectrograms within a hybrid model and reported competitive accuracy on a limited-

vocabulary keyword-recognition benchmark [55]. In both cases the quantum component is small and evaluated in simulation.

The Fourier perspective also suggests a property that has received less attention: a fixed spectrum acts as a built-in band limit. A model whose spectrum is $\{-K, \dots, K\}$ cannot represent signal components above frequency K, so it cannot fit high-frequency noise outside its band, regardless of how the parameters are trained. This gives a concrete answer to the question raised in the Introduction: a predetermined spectrum is an advantage when the designer knows where the signal lives. Noise within the band can still be fitted, and the interplay between spectrum design, overfitting, and generalization in quantum Fourier models is subtle [56].

5.4 Scientific Computing and Differential Equations

Spectral methods, which represent the solution of a differential equation as a truncated Fourier or Chebyshev series, are among the most accurate numerical techniques for problems with smooth solutions [57]. A QNN whose output is a truncated Fourier series can therefore be viewed as a trainable spectral solver. Physics-informed neural networks (PINNs) train a network to satisfy a differential equation by including the equation's residual in the loss [58], and the same strategy applies to QNNs. Because the QNN output is an analytic function of x, its input derivatives can be computed exactly, without finite differences.

Kyriienko, Paine, and Elfving developed this idea as *differentiable quantum circuits* (DQCs): functions are represented as expectation values of parameterized circuits, their derivatives are obtained analytically, and the circuits are trained to satisfy the equation and its boundary conditions [59]. They introduced a Chebyshev feature map and applied the method to nonlinear problems, including an instance of the Navier–Stokes equations for flow in a convergent–divergent nozzle. The Chebyshev map illustrates how the Fourier picture can be adapted to non-periodic domains: encoding $\varphi(x) = \arccos x$ instead of x turns $\cos(k\varphi)$ into the Chebyshev polynomial $T_k(x)$, so the Fourier series in φ becomes a Chebyshev series in x. Jaderberg et al. subsequently made the encoding generators trainable, allowing the model to adapt its own frequencies, and reported improved accuracy on the Navier–Stokes problem with only one additional parameter per encoding gate [43].

Scientific computing fits the criteria well when the domain is low-dimensional and the solution is smooth or oscillatory, as in wave propagation and problems with periodic boundary conditions; problems in three spatial dimensions plus time, however, quickly encounter the exponential growth of the multivariate spectrum.

5.5 High-Dimensional Classical Data: Vision, Tabular Data, and Finance

The tasks that dominate classical deep learning, such as image classification and large-scale tabular prediction, satisfy the criteria least well. A 28×28 grayscale image is a 784-dimensional input: encoding every pixel requires hundreds of qubits or aggressive dimensionality reduction, and the resulting spectrum is astronomically large while the target, a class label, bears no obvious relation to its frequency structure.

Computer vision. Practical quantum vision models are therefore hybrid: quanvolutional networks apply small quantum circuits to local image patches, like convolutional filters [60], and transfer-learning approaches place a small QNN on top of a pretrained classical feature extractor [61]. A large-scale benchmark by Bowles, Ahmed, and Schuld, however, found that out-of-the-box classical models systematically outperformed prototypical quantum classifiers on small classification tasks, that hybrid models performed similarly to their classical hosts, and that removing entanglement often did not reduce performance [62]. Claims of advantage on standard vision benchmarks should therefore be treated with caution.

Tabular data and finance. Finance has been a prominent target for quantum machine learning [63]. Fraud detection illustrates both the opportunities and the constraints. Grossi et al. applied a quantum support vector machine, a kernel-based relative of the QNN, to real card-payment data and found that results depended strongly on feature selection; the quantum model contributed complementary information within a hybrid ensemble, but only on a drastically reduced dataset that fit the hardware of the time [64]. Fraud datasets are also large and highly imbalanced, which conflicts with the fourth criterion. The Fourier perspective nevertheless identifies a natural niche: *cyclic features*. Transaction timestamps carry hour-of-day and day-of-week patterns, and classical pipelines typically encode such features through sine and cosine transforms. A QNN represents periodic dependence on these features natively; for example, encoding $x = 2\pi h/24$ for hour h makes the model exactly periodic over a day. Relational structure in transaction networks can be addressed by quantum graph neural networks [65], although these face the same scaling constraints.

In summary, for high-dimensional classical data the realistic role of a QNN today is that of a small, specialized component within a hybrid model, operating on a compact set of features whose structure matches its inductive bias.

5.6 Quantum-Native Data

When the data are themselves quantum states, for instance the output of a quantum simulation, sensor, or physical experiment, no classical-to-quantum encoding is required and the obstacles associated with criteria 1 and 4 largely disappear. Cong, Choi, and Lukin introduced quantum convolutional neural networks and showed that they can recognize quantum phases of matter using a number of parameters that grows only logarithmically with system size [66]. Huang et al. proved, and demonstrated experimentally, that for certain tasks learning from quantum experiments can require exponentially fewer experiments when quantum data are processed coherently by a quantum computer than when they are first measured and processed classically [67]. In addition, quantum models have been shown to generalize from relatively few training examples [68]. For quantum data, the question of advantage rests on firmer theoretical ground than for any classical-data application discussed above.

5.7 Limitations and Outlook

Data loading and measurement cost. Many proposed quantum speedups assume that classical data can be loaded into quantum states efficiently, an assumption that often

Table 2: Suitability of QNNs across application domains, assessed against the criteria of Section 5.1.

Domain	Input dim.	Spectral structure	Fit	Main obstacle
Implicit neural representations	1–5	Rich but structured; edges are problematic	High	Number of coordinates; Gibbs oscillations at edges
Audio and time series	1–few	Known periodicities and harmonics	High	Broadband noise and non-stationarity
Differential equations	1–4	Smooth or oscillatory solutions	High	Multivariate spectrum growth in higher dimensions
Computer vision	10^2–10^6	Weakly related to the target	Low	Encoding cost; classical baselines are strong
Tabular data and finance	10–10^3	Mostly unstructured, except cyclic features	Medium	Large, imbalanced datasets
Quantum-native data	n/a	Outside the Fourier framework	Highest	Access to quantum data sources

dominates the true cost [69]. At the output, estimating an expectation value to precision ϵ requires on the order of $1/\epsilon^2$ circuit executions, so every prediction carries a sampling overhead that classical networks do not incur.

Trainability and noise. As discussed in Section 4.4, barren plateaus make gradients vanish exponentially in the number of qubits for sufficiently random circuits [31], and hardware noise can induce barren plateaus of its own [47]. Since the parameter-shift rule requires two circuit evaluations per parameter, the sampling overhead also grows with the number of parameters.

Constrained expressivity. A large spectrum does not imply that all coefficients are freely adjustable. Mhiri et al. showed that the variance of a Fourier coefficient depends on the redundancy of its frequency, that is, on how many pairs of eigenvalue sums produce it, and that some coefficients can vanish exponentially as the number of qubits grows [70]. For uniform Pauli encodings, redundancy is highest at low frequencies, which suggests that QNNs can exhibit a form of spectral bias of their own. Exponential encodings [71, 72] reduce the number of gates needed for a given spectrum but also reduce redundancy, a trade-off that should be assessed explicitly.

Classical surrogates and dequantization. The Fourier structure that makes QNNs interpretable also makes them vulnerable to classical approximation. If the spectrum is known, a classical model can fit the same function class, either by constructing a classical surrogate from the quantum model [73] or by sampling frequencies with random Fourier features [33]. Sweke et al. characterized the conditions under which random Fourier features efficiently dequantize variational QML for regression [74]. Consequently, the applications most worth pursuing are those in which the relevant spectrum is hard to sample classically

or the data are quantum-native.

Outlook. Table 2 summarizes the analysis. The domains with the best fit share low-dimensional inputs, structured spectral content, and moderate data volume: INRs, periodic signal modeling, and spectral solvers for differential equations are the most promising classical-data applications, and quantum-native data the most promising overall. For high-dimensional classical data, QNNs are best viewed as specialized components within hybrid architectures. The Fourier perspective thus offers more than a description of what QNNs compute; it provides a principled way to decide where they should be applied.

6 Conclusion

This white paper has developed quantum neural networks from first principles, with the aim of making their behavior verifiable step by step for readers from either deep learning or quantum computing. Working through a three-qubit circuit explicitly, we showed that angle encoding produces a product state whose amplitudes are products of half-angle sines and cosines, that a trainable rotation about the same axis as the encoding acts as a bias rather than a weight, that CNOT gates permute amplitudes and thereby carry information from control to target qubits, and that measurement turns the quantum state into a real-valued output through an expectation value.

These observations combine into the central result of the paper: the output of a QNN with Hamiltonian-generated encodings is a finite Fourier series whose frequencies are fixed by the eigenvalues of the encoding Hamiltonians and whose coefficients are set by the trainable parameters. Expressivity is therefore governed primarily by the encoding strategy. Parallel encoding and data re-uploading yield identical spectra for the same number of encoding gates and differ only in whether they consume qubits or circuit depth, while input scaling relocates frequencies without adding gates, subject to a budget of at most 3^N frequencies for N encoding gates. The same Fourier structure yields exact gradients through the parameter-shift rule, although barren plateaus and sampling costs remain serious obstacles to training at scale.

Returning to the question posed in the Introduction, a predetermined spectrum is both a limitation and an advantage. It prevents a QNN from representing any frequency outside Ω, but it also gives the designer explicit, interpretable control over the spectral inductive bias of the model, including a built-in band limit. This makes QNNs best suited to low-dimensional problems with known or structured spectral content, such as implicit neural representations, periodic signals, and differential equations, and, most promisingly, to quantum-native data. The same structure, however, exposes QNNs to classical surrogates and dequantization by random Fourier features. Establishing a practical advantage on classical data will therefore require tasks whose relevant spectrum is hard to reproduce classically, together with hardware that can execute the necessary circuits reliably. We hope that the explicit derivations presented here provide a foundation for evaluating such claims.

References

[1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.

[2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[6] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.

[7] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

[8] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[10] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[11] Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2022.

[12] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019.

[13] International Energy Agency. Key questions on energy and AI. Technical report, IEA, Paris, 2026.

[14] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6/7):467–488, 1982.

[15] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[16] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219, 1996.

[17] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[18] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[19] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.

[20] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638(8052):920–926, 2025.

[21] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.

[22] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549:195–202, 2017.

[23] Maria Schuld and Francesco Petruccione. *Machine Learning with Quantum Computers*. Springer, 2nd edition, 2021.

[24] Edward Farhi and Hartmut Neven. Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002*, 2018.

[25] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98(3):032309, 2018.

[26] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019.

[27] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019.

[28] Maria Schuld and Nathan Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019.

[29] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020.

[30] Amira Abbas, David Sutter, Christa Zoufal, Aurélien Lucchi, Alessio Figalli, and Stefan Woerner. The power of quantum neural networks. *Nature Computational Science*, 1:403–409, 2021.

[31] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018.

[32] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R. McClean. Power of data in quantum machine learning. *Nature Communications*, 12:2631, 2021.

[33] Jonas Landman, Slimane Thabet, Constantin Dalyac, Hela Mhiri, and Elham Kashefi. Classically approximating variational quantum machine learning with random Fourier features. In *International Conference on Learning Representations (ICLR)*, 2023.

[34] M. Cerezo, Martin Larocca, Diego García-Martín, N. L. Diaz, Paolo Braccia, Enrico Fontana, Manuel S. Rudolph, Pablo Bermejo, Aroosa Ijaz, Supanut Thanasilp, Eric R. Anschuetz, and Zoë Holmes. Does provable absence of barren plateaus imply classical simulability? *Nature Communications*, 16:7907, 2025.

[35] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021.

[36] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5301–5310. PMLR, 2019.

[37] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.

[38] Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[39] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[40] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[41] Matus Telgarsky. Benefits of depth in neural networks. In *29th Annual Conference on Learning Theory*, volume 49 of *Proceedings of Machine Learning Research*, pages 1517–1539. PMLR, 2016.

[42] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. Elementary gates for quantum computation. *Physical Review A*, 52(5):3457–3467, 1995.

[43] Ben Jaderberg, Antonio A. Gentile, Youssef Achari Berrada, Elvira Shishenina, and Vincent E. Elfving. Let quantum neural networks choose their own frequencies. *arXiv preprint arXiv:2309.03279*, 2023.

[44] Elias M. Stein and Rami Shakarchi. *Fourier Analysis: An Introduction*, volume 1 of *Princeton Lectures in Analysis*. Princeton University Press, 2003.

[45] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331, 2019.

[46] M. Cerezo, Akira Sone, Tyler Volkoff, Lukasz Cincio, and Patrick J. Coles. Cost function dependent barren plateaus in shallow parametrized quantum circuits. *Nature Communications*, 12:1791, 2021.

[47] Samson Wang, Enrico Fontana, M. Cerezo, Kunal Sharma, Akira Sone, Lukasz Cincio, and Patrick J. Coles. Noise-induced barren plateaus in variational quantum algorithms. *Nature Communications*, 12:6961, 2021.

[48] Edward Grant, Leonard Wossnig, Mateusz Ostaszewski, and Marcello Benedetti. An initialization strategy for addressing barren plateaus in parametrized quantum circuits. *Quantum*, 3:214, 2019.

[49] Berta Casas and Alba Cervera-Lierta. Multidimensional Fourier series with quantum circuits. *Physical Review A*, 107:062612, 2023.

[50] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 165–174, 2019.

[51] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421, 2020.

[52] Jiaming Zhao, Wenbo Qiao, Peng Zhang, and Hui Gao. Quantum implicit neural representations. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 60940–60956. PMLR, 2024.

[53] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021.

[54] Dimitrios Emmanoulopoulos and Sofija Dimoska. Quantum machine learning in finance: Time series forecasting. *arXiv preprint arXiv:2202.00599*, 2022.

[55] Chao-Han Huck Yang, Jun Qi, Samuel Yen-Chi Chen, Pin-Yu Chen, Sabato Marco Siniscalchi, Xiaoli Ma, and Chin-Hui Lee. Decentralizing feature extraction with quantum convolutional neural network for automatic speech recognition. In *2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6523–6527, 2021.

[56] Evan Peters and Maria Schuld. Generalization despite overfitting in quantum machine learning models. *Quantum*, 7:1210, 2023.

[57] John P. Boyd. *Chebyshev and Fourier Spectral Methods*. Dover Publications, 2nd edition, 2001.

[58] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[59] Oleksandr Kyriienko, Annie E. Paine, and Vincent E. Elfving. Solving nonlinear differential equations with differentiable quantum circuits. *Physical Review A*, 103:052416, 2021.

[60] Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, and Tristan Cook. Quanvolutional neural networks: Powering image recognition with quantum circuits. *Quantum Machine Intelligence*, 2:2, 2020.

[61] Andrea Mari, Thomas R. Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340, 2020.

[62] Joseph Bowles, Shahnawaz Ahmed, and Maria Schuld. Better than classical? The subtle art of benchmarking quantum machine learning models. *arXiv preprint arXiv:2403.07059*, 2024.

[63] Dylan Herman, Cody Googin, Xiaoyuan Liu, Yue Sun, Alexey Galda, Ilya Safro, Marco Pistoia, and Yuri Alexeev. Quantum computing for finance. *Nature Reviews Physics*, 5:450–465, 2023.

[64] Michele Grossi, Noelle Ibrahim, Voica Radescu, Robert Loredo, Kirsten Voigt, Constantin von Altrock, and Andreas Rudnik. Mixed quantum–classical method for fraud detection with quantum feature selection. *IEEE Transactions on Quantum Engineering*, 3:1–12, 2022.

[65] Guillaume Verdon, Trevor McCourt, Enxhell Luzhnica, Vikash Singh, Stefan Leichenauer, and Jack Hidary. Quantum graph neural networks. *arXiv preprint arXiv:1909.12264*, 2019.

[66] Iris Cong, Soonwon Choi, and Mikhail D. Lukin. Quantum convolutional neural networks. *Nature Physics*, 15:1273–1278, 2019.

[67] Hsin-Yuan Huang, Michael Broughton, Jordan Cotler, Sitan Chen, Jerry Li, Masoud Mohseni, Hartmut Neven, Ryan Babbush, Richard Kueng, John Preskill, and Jarrod R. McClean. Quantum advantage in learning from experiments. *Science*, 376(6598):1182–1186, 2022.

[68] Matthias C. Caro, Hsin-Yuan Huang, M. Cerezo, Kunal Sharma, Andrew Sornborger, Lukasz Cincio, and Patrick J. Coles. Generalization in quantum machine learning from few training data. *Nature Communications*, 13:4919, 2022.

[69] Scott Aaronson. Read the fine print. *Nature Physics*, 11:291–293, 2015.

[70] Hela Mhiri, Léo Monbroussou, Mario Herrero-González, Slimane Thabet, Elham Kashefi, and Jonas Landman. Constrained and vanishing expressivity of quantum Fourier models. *Quantum*, 9:1847, 2025.

[71] S. Shin, Y. S. Teo, and H. Jeong. Exponential data encoding for quantum supervised learning. *Physical Review A*, 107:012422, 2023.

[72] Mo Kordzanganeh, Pavel Sekatski, Leonid Fedichkin, and Alexey Melnikov. An exponentially-growing family of universal quantum circuits. *Machine Learning: Science and Technology*, 4(3):035036, 2023.

[73] Franz J. Schreiber, Jens Eisert, and Johannes Jakob Meyer. Classical surrogates for quantum learning models. *Physical Review Letters*, 131:100803, 2023.

[74] Ryan Sweke, Erik Recio-Armengol, Sofiene Jerbi, Elies Gil-Fuster, Bryce Fuller, Jens Eisert, and Johannes Jakob Meyer. Potential and limitations of random Fourier features for dequantizing quantum machine learning. *Quantum*, 9:1640, 2025.